

PlenumHub / Polyxan

Toward a Source-First Publishing Substrate

Flyxion
Independent Researcher

July 2026 (revised August 2026)

Abstract

This essay outlines a proposed publishing architecture within the PLENUMHUB/POLYXAN/Spherepop lineage, one that treats structured source material rather than any rendered artifact as canonical. Video, audio, prose at varying depths, comics, and interactive renderings are all generated views of an underlying, addressable, branchable state history. The essay argues that most of the epistemic and practical failures of contemporary video-based publishing — oversimplification, unaccountable pronouncement, uncorrectable error, loss of provenance — are consequences of a specific and correctable design decision: conflating source and rendering into a single frozen artifact. It situates this claim within the broader stack, in which Spherepop supplies event histories and provenance, Polyxan supplies addressable semantic and hypermedia structure, and PlenumHub supplies coordination and governance — and argues that source-first publishing is not a new addition to that stack but a straightforward instance of its existing commitments.

To make this claim precise rather than merely evocative, the essay develops a typed calculus of semantic objects, transformation rules, closure, entropy-bounded merge, and sheaf-style coherence, and proves that well-typedness is preserved under revision, that erasure is eliminated by construction, and that no rendering pipeline can ever be fully faithful to an unboundedly long history — a limit the architecture cannot remove but can make reversible rather than permanent through retained provenance and progressive disclosure. It then turns the same scrutiny on itself, showing where the calculus's guarantees are unconditional and where they depend on assumptions — computable entropy, deterministic renderers, retained history, an independently checkable rule set — that a real deployment must supply rather than inherit for free, and naming three questions the substrate does not and cannot answer on its own: which branch a reader sees by default, whether an inference attached to a citation was ever recorded, and whether a rendering pipeline invents content it cannot actually produce. Along the way it formalizes what a reader is entitled to demand of such a system, why addressable identity is a precondition for citation and discourse rather than a bookkeeping convenience, and why retaining history rather than a mere current snapshot is what makes forgetting reversible instead of final.

This revision extends the essay in five further directions: it argues that artificial ingestion limits are a misplaced choke point relative to the genuine costs of preservation, availability, and physical realization; it gives citation a full computational content — an executable citation as a typed, verifiable derivation rather than a pointer — and uses it to make the substrate's own audit machinery concrete rather than aspirational; it strengthens two previously acknowledged but underdeveloped failure modes (provenance overload, renderer concentration) using that same machinery; and it closes by situating the proposal against a companion operating-system specification, Spherepop OS, showing the two documents' independently-derived formalisms converge strongly while surfacing at least one genuine correction each document owes itself. It closes by proposing a public, competitive practice — a form of *pipeline golf* — in which participants compete to produce the most interesting, entertaining, or explanatorily faithful transformations of a fixed text into image, animation, or sound.

1 The Conflation Problem

Traditional publishing systems, digital and physical alike, generally conflate two things that need not be conflated: the *source* of a work and its *rendering*. A printed book had to be both, because there was no practical channel for distributing structure separately from its typeset form. Contemporary video platforms inherited this fusion by default rather than by necessity. A YouTube video is simultaneously the argument and the recording of the argument; there is no accessible, addressable, versioned object underneath it that a viewer, an editor, or the author's future self can revise, diff, branch, or query.

This conflation produces several downstream pathologies. A mistake in a video is *uncorrectable*: it is permanent unless the entire artifact is withdrawn and re-recorded, and even then, prior copies persist independently in caches and reposts. Because playback is time-locked rather than reader-paced, the medium imposes a kind of *forced linearity*: caveats, branches, and counterexamples — the very content that often carries the most information in theoretical work — are systematically pruned in favor of singular, closure-seeking narratives. There is a *loss of provenance*, since two versions of a talk are unrelated artifacts with no diff between them; there is no equivalent of tracked changes or an errata sheet. And because the medium resists rebuttal-in-kind, an *asymmetry of authority* sets in: claims — particularly interpretive claims about an audience's own psychology — become effectively unfalsifiable within the format that delivered them.

None of these follow from the use of speech or moving images as such. They follow from treating the rendering as the master copy.

2 The Core Inversion

The proposed correction is structural, not stylistic: *the canonical layer is the source; every video, audio track, comic panel, and prose summary is a compiled view of it*. This is the relationship that already holds, uncontroversially, between source code and a compiled binary, between a musical score and a performance, or between a statute's text and its citations. Publishing platforms built around audiovisual media are nearly alone in inverting this relationship.

2.1 Canonical layer

The canonical layer consists of a state or event graph of structured, addressable, diffable units of content, together with the definitions, references, and metadata attached to those units, and a full revision history that is retained rather than overwritten.

2.2 Derived layer

The derived layer consists of everything generated from that canonical layer: prose at variable depth, whether full, medium, or short; audio narration; video, at variable pacing and visual style; comics, slide decks, and interactive simulations; and machine-generated transcripts and descriptions of any of the above, closing the loop back to text.

Correction, branching, and merging all happen exclusively in the canonical layer. Every derived artifact is disposable and regenerable; none of them needs to be hand-edited, because none of them is authoritative.

3 Three Independent Axes

A recurring error in early sketches of this system is collapsing several genuinely independent dimensions into a single notion of “version.” At minimum, three axes must be tracked separately.

At minimum, three axes must be tracked separately. The first is *lineage*: revisions, meaning-preserving corrections or extensions written $N \rightarrow N + 1$, as distinct from branches, divergent and mutually non-superseding continuations from a common ancestor. Lineage is therefore not a line but a graph. The second is *depth*: deliberately lossy compression of a fixed lineage point — full, medium, short — independent of how recent or how correct that point is. A shorter rendering is not a newer one; a newer revision is not necessarily a shorter one. The third is *rendering*: the output modality, whether video, audio, comic, prose, or transcript, applied to a fixed lineage-and-depth coordinate.

A well-formed request to the system is therefore a coordinate across all three axes: “*the short video of the branch in which the star becomes a planet, as of revision 8.*” Treating these as one axis — as conventional platforms implicitly do, where “newer” and “shorter” and “different medium” are all just “another upload” — is precisely what makes existing systems hard to search, cite, or reason about.

4 Toward an Event Grammar

The state graph is only as useful as the precision of its units. Natural-language description — “the yellow triangle transforms into a green star” — is legible to humans but leaves duration, interpolation, persistence, and causation unspecified, and is therefore poorly suited to serve as the actual canonical layer. A structured event, by contrast, is machine-precise and diffable:

```
event {  
  target: object_1  
  time: 12.0  
  shape: star  
  color: green  
  transition: morph  
  duration: 2.0  
}
```

Prose, in this architecture, is *generated from* the event, not the reverse: “At 12 seconds, the yellow triangle morphs into a green star over two seconds.” The animated-shape case is a useful minimal illustration because its event grammar is nearly trivial to specify by hand. What such toy examples leave open, at the level of this essay, is what an analogous event grammar looks like for discursive, definition-driven theoretical prose, where the “events” are not spatial transformations but the introduction, refinement, or contrast of distinctions.

This is not, in fact, an unsolved problem for the wider program of which this essay is a part. Section 12 develops the necessary machinery directly: a typed calculus of semantic transformations, a closure operator guaranteeing that renderings are always recoverable from a source, an entropy-bounded merge operator for reconciling branches, and a coherence condition guaranteeing that locally acceptable renderings do not silently contradict one another when assembled globally.

5 Provenance Without Rigidity

A system that makes correction too easy risks destroying the very citability it was meant to preserve. The general problem of tracking where a piece of data came from and the process by which it arrived at its current form is well studied under the name data provenance [8]; this essay borrows the term and narrows it to the specific case of publishing, where the units being tracked are claims and renderings rather than database views. If a document’s identity is untethered from any fixed point — if paraphrase silently re-anchors what a given reference resolves to — then nothing can be quoted, cited, or held stable against later revision. The intended design resolves this the way version-control systems resolve it: a human-facing name (“the essay,” “Document A”) always resolves to a current state, but every prior state through which the document has passed remains individually visible and addressable in an unbroken list. Nothing is overwritten. “Show me the full text as it existed before the last revision” must always be answerable.

6 Why Histories, Not Snapshots?

Section 5 argued that correction must not untether a document’s identity from a fixed point, but it did not yet say why the architecture insists on retaining a full history rather than simply keeping an accurate current snapshot. This section states that reason directly, since the rest of the essay assumes it without pausing to justify it again.

A snapshot answers the question of what currently exists. A history answers the question of how the current state came to exist. These are not the same question, and a system that retains only the answer to the first has, by construction, discarded the answer to the second — which is exactly the erasure this essay has been arguing against since Section 1.

Definition 6.1 (Snapshot Map). Define $\pi(\sigma) = M$, the function taking a semantic object to its populated content map alone, discarding I , E , and S .

Proposition 6.2 (Snapshots Underdetermine History). π is non-injective: there exist $\sigma_1 = (I_1, T, M, E_1, S_1)$ and $\sigma_2 = (I_2, T, M, E_2, S_2)$ with $S_1 \neq S_2$ and $\pi(\sigma_1) = \pi(\sigma_2)$.

Proof. Let M be any fixed content map populated by a single modality k with content c . Let S_1 be the provenance graph of a single `introduce_term` event directly producing c , and let S_2 be the provenance graph of a chain `introduce_term`, `qualify`, `qualify` whose final rendering, by coincidence of template collapse, also produces c — for instance because the qualifications were later judged redundant with the base claim and contributed no additional surface text under the renderer in use. Then $\sigma_1 = (I_1, T, M, E_1, S_1)$ and $\sigma_2 = (I_2, T, M, E_2, S_2)$ satisfy $\pi(\sigma_1) = M = \pi(\sigma_2)$ while $S_1 \neq S_2$, witnessing non-injectivity. $\square$

Equivalently, and more directly connected to the illusion result of Theorem 12.24: $\pi(\sigma_1) = \pi(\sigma_2) \not\Rightarrow S_1 = S_2$. A snapshot cannot, even in principle, distinguish between two histories that happen to converge on the same current content, and a system that keeps only $\pi(\sigma)$ has thrown away exactly the information that would let it. This is the precise sense in which “how did it become this” is not recoverable from “what exists,” and it is why the architecture developed in Section 12 retains S as a first-class component of σ rather than treating it as an optional log a deployment could reasonably discard.

7 From Publishing System to Media Operating System

Once audience participation is admitted — suggestions, alternate continuations, competing branches — the model stops being a document with history and becomes a *history space*: a graph

rather than a line, traversed rather than read. A video renderer follows one path through the graph; a comic renderer may follow another; an exploratory renderer may expose the branching structure itself rather than collapsing it. This is the point at which the system converges, structurally, with source control, game engines, screenplay systems, and knowledge graphs — not by analogy, but because all of these are prior solutions to the same underlying problem: separating representation from rendering.

8 Querying Histories Instead of Reading Artifacts

The source-first inversion proposed throughout this essay changes not only how content is stored but how it is encountered. Conventional publishing systems assume that the primary interaction between a reader and a body of work is the act of reading a completed artifact. A reader opens a document, watches a video, or listens to a recording. Whatever history produced that artifact remains secondary, if it is visible at all.

A canonical-source architecture reverses this assumption. Because the source exists as a structured, addressable history rather than a frozen object, the natural unit of interaction becomes a query rather than a document. A reader may still choose to consume a rendered artifact from beginning to end, but that artifact is only one view among many. The underlying object remains available for inspection, traversal, and transformation.

This shift is easiest to see by considering questions that conventional documents answer poorly. A reader may wish to know when a particular definition entered a work, which revision introduced a controversial claim, which branches disagree about a conclusion, or which assumptions changed between two versions of the same argument. In a document-centered architecture these questions require manual reconstruction. In a source-first architecture they are direct queries against provenance: a request for every revision introducing a given definition, or every branch that disagrees about a given claim, or every rendering derived from a given event, is a traversal of S in the sense of Definition 12.1, not a search through prose.

The distinction resembles the difference between browsing a rendered webpage and querying a database. A webpage presents a fixed arrangement chosen in advance by its author. A database exposes a space of possible questions, many of which the author never anticipated. Source-first publishing extends this principle from data management to knowledge production. The primary object is not a narrative sequence but a structured history capable of supporting many narrative sequences.

This does not eliminate the value of authored presentations. A carefully written essay remains a useful path through a space of possibilities. What changes is its status. The essay becomes one traversal of the source rather than the source itself. Readers who wish to follow the author's path may do so. Readers who wish to inspect alternatives, recover omitted context, or examine the history that produced a conclusion are no longer constrained by the limits of the rendering.

Seen this way, the proposal is not merely a version-control system for documents. It is a transition from artifact-centric publishing to query-centric publishing. Documents remain possible, but they become answers to questions rather than containers of truth.

9 Reader Views and Reader Rights

Section 8 argued that the natural unit of interaction in a source-first system is a query rather than a document. This section makes explicit what a reader is entitled to query for, since a query-centric architecture is only as useful as the space of admissible queries it actually supports.

A reader encountering a rendering $M(k)$ of some σ is not limited to accepting that rendering at face value. At minimum, six distinct requests should be treated as first-class operations rather than incidental features of a particular interface. A reader may request the provenance behind a claim, resolving to the chain of rule applications that produced it, in the sense of Definition 12.1. A reader may request the alternatives to a given branch, surfacing sibling continuations that share a common ancestor but were never merged, per Definition 12.11. A reader may request the disagreements among branches, which is a request for the specific point at which two provenance graphs diverge rather than a request for either graph in full. A reader may request the source itself, at whatever depth the underlying object supports, rather than being confined to the depth a particular renderer chose to produce. A reader may request the rendering path connecting a source to a given output, which Section 14 develops in depth as the executable citation. And a reader may request the context a shorter rendering omitted, which Proposition 12.28 shows is always answerable in principle, provided the underlying histories remain retained.

None of these six requests introduces new formal machinery beyond what Section 12 already defines; each is simply a named traversal of σ that a conventional document-centric interface has no vocabulary for expressing. The point of naming them here, before the calculus itself is developed, is to record what a source-first architecture owes its readers independently of how any particular implementation chooses to expose it. A system that retains full provenance but never surfaces these six requests to a reader has built the substrate without honoring what the substrate was for.

It is worth being precise about what this list does not claim. It does not claim that every reader will exercise these rights, nor that exercising them is costless — Section 17 discusses the countervailing risk of provenance overload directly. It claims only that the architecture should make the six requests possible, leaving whether and how often a given reader chooses to use them as a separate question about interface design and reader disposition, not about what the substrate permits.

10 Pipeline Golf

A subsidiary and more immediately playable component of this vision is a public competitive practice, in the spirit of vim golf or code golf, applied to text-to-image and text-to-video pipelines. A fixed source text — a short passage, a single paragraph, an event sequence — is given to participants, who each construct a distinct rendering pipeline for it. Submissions are judged not on a single axis but several, potentially including:

Submissions are judged not on a single axis but several, potentially including fidelity — how much of the source’s structure survives the transformation; economy — how few steps, tokens, or how little compute the pipeline requires; explainability — how legible the pipeline’s own logic is to an outside reader, whether someone else can read the pipeline and understand why it produced what it produced; and entertainment or surprise — whether the resulting rendering is interesting independent of its fidelity, in the way an unusual translation or an eccentric adaptation can be valuable precisely for departing from its source.

This practice serves two purposes beyond its own entertainment value. First, it is a standing, low-stakes proving ground for the rendering layer of the larger system, generating a continuously growing library of pipelines against which the canonical-source architecture can be tested. Second, it directly operationalizes the essay’s central claim: that a single source can and should support many legitimate, non-competing renderings, and that the interesting design space lies in the diversity and quality of the transformation, not in the enforcement of one official version.

Pipeline Golf is not part of the substrate developed in Section 12, and no theorem in this essay depends on it. It is a cultural practice intended to drive renderer development the way vim golf and code golf drive tooling development for their own domains: a source of demonstration pipelines, not a source of proof obligations. Judging a submission by fidelity, economy, explainability, or surprise is a social convention this essay recommends, not a scoring function derivable from Definition 12.9 or any other part of the calculus; a future version of the practice could adopt an entropy-based score without changing anything the calculus already proves, but nothing here requires it to. Section 17.1’s salience weighting supplies exactly such an entropy-based score, should the practice choose to adopt one: a submission’s fidelity-per-bit could be scored by how little it raises E relative to its source, at no cost to the calculus’s own proofs.

11 Canonical Sources and Educational Media

The architecture developed in the remainder of this essay is easiest to evaluate against a concrete domain rather than in the abstract, and education is an unusually clean fit for reasons worth making explicit before the formal apparatus of Section 12 begins.

A textbook chapter, a recorded lecture, a set of flashcards, and an interactive simulation covering the same topic are, under ordinary publishing practice, four independent artifacts maintained separately, correctable only by separately editing each one, and prone to drifting out of agreement with each other as errors are fixed in one but not the others. Under the architecture this essay proposes, they are four renderings of a single σ whose declared modalities are PROSE-FULL, VIDEO, FLASHCARD-SET, and SIMULATION, in the sense of Definition 12.1. A correction to the underlying claim — a fixed date, a corrected derivation, an updated example — is made once, to the source, and the closure operator of Definition 12.7 regenerates whichever renderings a course or a learner is actually using. This is not a new capability invented for education; it is the general claim of Section 2 applied to a domain where the cost of drift between renderings is unusually visible, since a student working from an outdated flashcard set while a lecture has already been corrected is a failure mode instructors already recognize by other names.

Education also stresses the depth axis of Section 3 in a way general-purpose publishing does not. The same source plausibly needs a one-sentence definition, a worked example, a full derivation, and a set of practice problems, and a learner’s need for each depends on where they are in a course rather than on any property of the material itself. Progressive disclosure, in the sense of Section 12.7, is close to a description of how a well-designed course already tries to sequence material; the contribution here is only to make the underlying object explicit enough that the sequencing can be generated on demand rather than authored separately for every cohort.

None of this argues that educational content is where the architecture is easiest to build — the entropy-measurement difficulties of Section 13.2 and the event-grammar limitations of Section 13.1 apply here exactly as they apply elsewhere. What education offers is not an easier case but a more legible one: a domain where the cost of the conflation problem diagnosed in Section 1 is already familiar to anyone who has taught from materials that fell out of sync with each other, which makes the proposal feel like a description of an existing frustration rather than a speculative architecture in search of a use case.

12 A Formal Substrate for Canonical Sources

This section develops, from first principles and without appeal to outside material, the minimal formal machinery needed to make the claims of Sections 1–5 precise: that a canonical source

can be defined independently of any rendering, that every rendering is recoverable from it, that branches can be reconciled rather than merely coexist, and that the collection of all admissible renderings of a source cannot silently contradict itself as it scales.

12.1 Semantic Objects

Definition 12.1 (Semantic Object). A semantic object is a tuple $\sigma = (I, T, M, E, S)$ where I is an immutable identity, T is a finite set of required modality tags (e.g. PROSE-FULL, PROSE-SHORT, AUDIO, VIDEO, COMIC), $M : T \rightarrow \text{Content}$ is a partial content map assigning payloads to modalities, $E \in \mathbb{R}_{\geq 0}$ is a scalar entropy value, and S is a finite, acyclic provenance graph recording the derivation history of σ .

Definition 12.2 (Well-Typedness). σ is *well-typed*, written $\sigma \vdash \text{valid}$, if $\text{dom}(M) \subseteq T$: every populated modality is one of the declared required modalities. Note that σ may be well-typed while M is only partially defined on T ; well-typedness constrains what may be populated, not what must be.

This single distinction — *declared* modalities T versus *populated* modalities $\text{dom}(M)$ — is what separates a canonical source from a rendering. A source declares that it should eventually support text, audio, and video; a particular saved state of that source may have only text populated. The gap between T and $\text{dom}(M)$ is precisely the space of renderings still owed to the reader.

12.2 Typed Transformations

Definition 12.3 (Rule). A rule is a triple $r : a \rightsquigarrow b$ with entropy budget $\epsilon_r \geq 0$, where a, b are modality tags. Applying r to σ is valid exactly when $a \in \text{dom}(M)$, and produces

$$\sigma \xrightarrow{r} \sigma' = (I, T, M[b \mapsto r(M(a))], E', S \cup \{r\}),$$

subject to the entropy guard $E' \leq E + \epsilon_r$.

A rule from PROSE-FULL to VIDEO is a video renderer; a rule from PROSE-FULL to PROSE-SHORT is a summarizer; a rule from EVENT to PROSE-FULL is exactly the “prose is generated from structured events” claim of Section 4, instantiated as a concrete morphism rather than a metaphor.

Theorem 12.4 (Type Preservation). *If $\sigma \vdash \text{valid}$ and $\sigma \xrightarrow{r} \sigma'$ is a valid rule application, then $\sigma' \vdash \text{valid}$.*

Proof. By definition of validity, applying $r : a \rightsquigarrow b$ requires $a \in \text{dom}(M) \subseteq T$. The resulting content map is $M' = M[b \mapsto r(M(a))]$, so $\text{dom}(M') = \text{dom}(M) \cup \{b\}$. Since σ is well-typed we have $\text{dom}(M) \subseteq T$; it remains to check $b \in T$. A rule is only declared usable on σ if its target modality is among the required set — this is a side condition on rule declaration, not on application — so $b \in T$ by hypothesis on the rule’s admissibility for σ . Hence $\text{dom}(M') \subseteq T$, and $\sigma' \vdash \text{valid}$. $\square$

This is the formal content of “every rendering is a projection of the source, not a separate object”: no sequence of rule applications can ever produce a state that has escaped the declared type of the object it was generated from. A video generated from an essay is, by construction, still typed as a rendering of that essay; there is no operation in this calculus that could quietly detach the video from its source and make it free-floating, which is exactly the property YouTube-style publishing lacks.

Corollary 12.5. *For any finite chain $R = r_1; r_2; \dots; r_n$ of valid rule applications, if $\sigma \vdash \text{valid}$ and $\sigma \Downarrow_R \sigma_n$, then $\sigma_n \vdash \text{valid}$.*

Proof. Immediate by induction on n using Theorem 12.4 as the inductive step, with the base case $n = 0$ trivial since $\sigma_0 = \sigma$. $\square$

12.3 Rendering as Closure

The closure operator defined below is only as strong as its ability to actually reach every declared modality, and that ability is not automatic. It depends on a renderer existing for each missing modality, which is an assumption about the rule set, not a fact about σ itself. Making this assumption an explicit object, rather than leaving it implicit in the word “declared,” is necessary before closure can be stated honestly.

Definition 12.6 (Reach). For a semantic object $\sigma = (I, T, M, E, S)$, define

$$\text{Reach}(\sigma) = \{k \in T : \text{there exists a valid rule chain from some } k' \in \text{dom}(M) \text{ to } k\}.$$

σ is *renderer-complete* if $\text{Reach}(\sigma) = T$.

Renderer-completeness is not guaranteed by well-typedness. A well-typed σ may declare a modality in T for which no rule targeting it has ever been written — a source that promises a video rendering but for which no text-to-video rule exists in the rule set is well-typed and not renderer-complete. This is precisely the situation Section 4 flagged for prose: a declared modality with no known path to it is not a defect in σ , it is an absence in the available rule set, and the two should not be confused.

Definition 12.7 (Closure). The closure operator Q is defined only on renderer-complete objects. For such σ , $Q(\sigma) = \sigma$ if $\text{dom}(M) = T$, and otherwise $Q(\sigma)$ is the result of applying, for every $k \in T \setminus \text{dom}(M)$, some declared rule targeting k from an already-populated modality, until $\text{dom}(M) = T$. Q is undefined on σ for which $\text{Reach}(\sigma) \neq T$.

Proposition 12.8 (Idempotence of Closure). For renderer-complete σ , $Q(Q(\sigma)) = Q(\sigma)$.

Proof. By definition, $Q(\sigma)$ has $\text{dom}(M) = T$. Applying Q again finds no $k \in T \setminus \text{dom}(M)$, since this set is empty, and so returns its input unchanged: $Q(Q(\sigma)) = Q(\sigma)$. $\square$

Restricting closure to renderer-complete objects changes what the idempotence proposition is a proof of. It is not a proof that every declared rendering can be recovered; it is a proof that, conditional on renderers already existing for every declared modality, recovery is well-behaved and terminates at a stable point. That conditional is doing real work. Whether renderer-completeness holds for a given T — whether, for instance, a faithful video renderer for dense theoretical prose actually exists as a declared rule — is exactly the open problem Section 13.1 shows is solved only for a bounded fragment of prose, and is otherwise unresolved. Closure is therefore best read as a statement about what happens once renderability is available, not as a proof that renderability is available.

Closure is the formal statement of Section 2’s “derived layer”: a closed object has every declared rendering populated, and nothing about closing an already-closed object does anything further. This is what licenses treating video, audio, and prose-at-depth as *disposable*, for renderer-complete objects. If a rendering is lost or discarded, it is simply an unclosed modality slot, recoverable by re-running Q ; nothing about the identity I , the required set T , or the provenance S depended on that rendering having been kept.

12.4 Entropy and the Bound on Drift

Definition 12.9 (Semantic Entropy). For σ with populated modalities $\{M(k)\}_{k \in \text{dom}(M)}$, define

$$E(\sigma) = \sum_{k \in \text{dom}(M)} H(M(k)) - \sum_{k \neq k'} \text{MI}(M(k), M(k')),$$

where H is the entropy of a single modality’s content and MI is the mutual information between pairs of modalities.

The subtracted mutual-information term penalizes redundancy: two renderings that say exactly the same thing in different media contribute little beyond their pairwise overlap, while two renderings that are individually coherent but jointly uninformative about one another are, in this accounting, evidence that at least one of them has drifted from the source it claims to represent.

Theorem 12.10 (Entropy Boundedness Under Rule Chains). *For a chain $R = r_1; \dots; r_n$ with $\sigma \Downarrow_R \sigma_n$,*

$$E(\sigma_n) \leq E(\sigma) + \sum_{i=1}^n \epsilon_{r_i}.$$

Proof. By induction on n . For $n = 0$, $\sigma_n = \sigma$ and the claim is trivial. For the inductive step, assume $E(\sigma_{n-1}) \leq E(\sigma) + \sum_{i=1}^{n-1} \epsilon_{r_i}$. By the entropy guard in the definition of rule application, $E(\sigma_n) \leq E(\sigma_{n-1}) + \epsilon_{r_n}$. Combining the two inequalities gives the claim. $\square$

This is the precise sense in which a source-first system cannot silently accumulate error the way repeated re-uploads or re-transcriptions of a video can: every transformation carries a declared, bounded entropy cost, and the total drift after any finite chain of renderings is bounded by the sum of those declared costs, not by however much distortion happened to occur in an uncontrolled process.

It can help, purely as illustration and not as an additional formal claim, to picture $E(\sigma)$ as tracking the state of a field rather than a static number: a quantity that a well-behaved rule application is required to hold level or push down, and that an undisciplined one is free to let rise unchecked. Nothing in Definition 12.9 or Theorem 12.10 depends on this picture, and the essay makes no claim that semantic drift is governed by anything resembling physical law; the entropy guard is a bound the calculus imposes by definition on ϵ_r , not a conservation principle the world imposes on meaning. The image is offered only because it is a natural way to hold the shape of the boundedness result in mind before returning, in Section 13.2, to the harder question of what E actually measures once symbolic content is left behind.

12.5 Merge and Branching

Definition 12.11 (Guarded Merge). For two objects σ_a, σ_b sharing a common ancestor in their provenance graphs, define the merge $\sigma_a \oplus \sigma_b = \sigma_m$ to be defined only when

$$E(\sigma_m) \leq \max(E(\sigma_a), E(\sigma_b)) + \epsilon_{\oplus}$$

for some fixed merge budget $\epsilon_{\oplus}$, with $M_m(k)$ reconciled from $M_a(k)$ and $M_b(k)$ by a declared reconciliation rule wherever both are populated, and inherited directly wherever only one is.

Merge is partial by design: two branches that disagree too sharply — whose reconciled entropy would exceed the guard — simply do not merge automatically, and are left as coexisting branches. This is the formal counterpart of Section 7’s claim that “neither is a revision of the other”: the calculus does not force a choice between competing continuations, it only defines the conditions under which such a choice becomes unnecessary because the continuations were compatible all along.

Proposition 12.12 (Merge Respects the Entropy Bound). *If $\sigma_a \oplus \sigma_b$ is defined, then $E(\sigma_a \oplus \sigma_b) \leq \max(E(\sigma_a), E(\sigma_b)) + \epsilon_{\oplus}$.*

Proof. Immediate: this inequality is the definedness condition of the merge operator itself, not a derived fact. A merge that would violate it is, by definition, not a value the operator produces. $\square$

Definition 12.13 (Merge Identity Policy). Definition 12.11 does not, by itself, specify $I(\sigma_a \oplus \sigma_b)$. A deployment must supply an explicit policy

$$I(\sigma_a \oplus \sigma_b) = \Phi_I(I(\sigma_a), I(\sigma_b))$$

for some function Φ_I external to the merge operator — not necessarily one that selects one of σ_a 's or σ_b 's existing identity, since a policy that mints a fresh identity for the merged object is equally consistent with everything Definition 12.11 states.

Proposition 12.14 (Merge Leaves Identity Genuinely Open). *Neither E , $\oplus$'s entropy guard, nor the reconciliation rule governing M_m constrains $I(\sigma_a \oplus \sigma_b)$.*

Proof. Immediate from Definition 12.11: every clause of the definition concerns M_m or the entropy bound on $E(\sigma_m)$; none references I . A merge satisfying every stated clause remains well-defined for any choice of Φ_I in Definition 12.13. $\square$

12.6 Coherence Across Local Views

The remaining requirement is the one this essay has been calling admissibility from Section 1 onward: that renderings generated locally — for one reader, one depth, one branch — must not silently contradict renderings generated for another, when both descend from the same source. This is a gluing condition, and it is naturally stated in the language of sheaves.

Definition 12.15 (Local View Assignment). Let $\mathcal{U}$ be a covering of some index set of contexts (readers, depths, branches) by subsets $\{U_i\}$. A local view assignment is a function G sending each U_i to the rendering of σ appropriate to that context, together with restriction maps $\rho_{U_i U_j} : G(U_i) \rightarrow G(U_j)$ whenever $U_j \subseteq U_i$.

Definition 12.16 (Coherence Condition). G is *coherent* if for every pair of overlapping contexts U_i, U_j ,

$$\rho_{U_i, U_i \cap U_j}(G(U_i)) = \rho_{U_j, U_i \cap U_j}(G(U_j)),$$

and if, whenever a family of local views agrees pairwise on all overlaps, there exists a unique global view $G(\bigcup U_i)$ restricting to each of them.

Theorem 12.17 (Coherence Under Determinism). *If (i) every rendering rule is a deterministic function of its source modality, and (ii) two contexts U_i, U_j derive their local views from the same closed object $Q(\sigma)$, then any local view assignment G satisfies the coherence condition.*

Proof. By (ii), $G(U_i)$ and $G(U_j)$ are both computed by applying declared rules to the same source $Q(\sigma)$. By (i), applying the same rule to the same input yields the same output; hence for any modality k populated in both contexts, $\rho_{U_i, U_i \cap U_j}(G(U_i))(k) = \rho_{U_j, U_i \cap U_j}(G(U_j))(k)$, since both sides are the deterministic image of $M(k)$ under the same rule. Agreement on overlaps established, uniqueness of the glued global view follows because a global assignment consistent with all local restrictions is exactly the restriction of $Q(\sigma)$ itself, and any two global assignments agreeing with every local one must agree with $Q(\sigma)$ pointwise, hence with each other. $\square$

Corollary 12.18. *If coherence fails — if two renderings of the same declared source disagree on their overlap — then by contraposition of Theorem 12.17, either the rules that produced them were not deterministic functions of a shared source, or the two renderings were not, in fact, derived from the same closed object. Either diagnosis is actionable: the first identifies a defective renderer, the second identifies an unauthorized fork masquerading as a rendering of the canonical source.*

This is the formal cash value of “renderings should not be mistaken for source.” Coherence failure is not merely an aesthetic inconvenience to be smoothed over; under this calculus it is a proof-theoretic signal that something has gone wrong upstream, and the corollary above tells you exactly what kind of thing to look for.

12.7 Progressive Disclosure

Several results already developed in this section, and one developed in the section that follows, share an unnamed pattern: a reader dissatisfied with a coarse rendering can ask for a finer one, and the finer rendering does not contradict the coarser one but refines it. This pattern deserves a name and a precise statement before the next subsection uses it to resolve a specific failure mode.

Definition 12.19 (Depth Order). Let $k_0, k_1, \dots, k_n$ be modalities corresponding to increasing depth in the sense of Section 3, and write $k_i \preceq k_j$ for $i \leq j$. The order is a *refinement order* if, for every pair $i \leq j$, there exists a declared rule $r : k_j \rightsquigarrow k_i$ — a deeper rendering can always be compressed down to a shallower one — and no declared rule in the reverse direction is guaranteed to exist.

Definition 12.20 (Progressive Disclosure). A rendering pipeline satisfies *progressive disclosure* with respect to a refinement order $k_0 \preceq \dots \preceq k_n$ if a reader may request $M(k_i)$ for any i without the pipeline requiring $M(k_j)$ for any $j > i$ to have been produced or inspected first.

Theorem 12.21 (Progressive Disclosure Resolves Distinguishability Monotonically). Let $k_0 \preceq \dots \preceq k_n$ be a refinement order, and let $S_1 \neq S_2 \in \mathcal{P}$ be provenance histories, in the sense of Definition 12.22, whose renderings at depth k_i are distinguishable: $\rho_{k_i}(S_1) \neq \rho_{k_i}(S_2)$, in the sense of Definition 12.23. If every rule $r : k_j \rightsquigarrow k_i$ guaranteed by Definition 12.19 is a well-defined function on the image of ρ_{k_j} , then $\rho_{k_j}(S_1) \neq \rho_{k_j}(S_2)$ for every $j > i$.

Proof. Suppose toward contradiction that $\rho_{k_j}(S_1) = \rho_{k_j}(S_2)$ for some $j > i$. By Definition 12.19, a declared rule $r : k_j \rightsquigarrow k_i$ exists, and $\rho_{k_i}(S_1) = r(\rho_{k_j}(S_1))$ and $\rho_{k_i}(S_2) = r(\rho_{k_j}(S_2))$ by construction of a rendering map as the composition of the rule chain producing it. Since $\rho_{k_j}(S_1) = \rho_{k_j}(S_2)$ by assumption, applying the same function r to equal inputs yields $r(\rho_{k_j}(S_1)) = r(\rho_{k_j}(S_2))$, hence $\rho_{k_i}(S_1) = \rho_{k_i}(S_2)$, contradicting the hypothesis that $\rho_{k_i}(S_1) \neq \rho_{k_i}(S_2)$. $\square$

The theorem is deliberately modest: it states that distinguishability at a shallow depth is never lost by moving deeper, not that moving deeper always gains distinguishing information the shallower depth lacked. That stronger claim is exactly Proposition 12.28 below, which is now visible as a special case of Theorem 12.21 applied to the specific pair of depths `SHORT` and `FULL`: illusion at a shallow depth is compatible with disambiguation at a deeper one because the refinement order guarantees the deeper rendering never contradicts the shallower one, only adds to it. Progressive disclosure, in this sense, is the general design principle of which “ask for more depth” is the specific instance this essay has relied on since Section 3 first distinguished depth from lineage and rendering.

12.8 The Limits of Any Rendering

The preceding subsections establish what a source-first architecture guarantees. This one establishes a limit no architecture of this kind, however carefully specified, can remove — and distinguishes that limit sharply from the pathology this essay opened by diagnosing.

The distinction developed here is not a fourth entry alongside the three axes of Section 3. Lineage, depth, and rendering are coordinates: they say which version of a source, compressed how much, in which modality, a request is asking for. Erasure and illusion, by contrast, are not coordinates but failure modes that can occur at any fixed choice of those coordinates, describing what has happened to the provenance behind a rendering rather than which rendering was requested. The two taxonomies meet at exactly one point, and it is worth naming in advance: Proposition 12.28 below shows that moving along the depth axis is the specific mechanism by which an instance of illusion, though never eliminated, can be resolved on demand. Depth is where the coordinate system and the failure mode intersect; lineage and rendering play no comparable role in disambiguation.

Definition 12.22 (Provenance Space). Let $\mathcal{P}$ denote the set of all provenance graphs reachable by finite chains of rule applications from some initial object, under Definition 12.3. Since a legal chain $R = r_1; \dots; r_n$ may have any finite length n , $\mathcal{P}$ is countably infinite.

Definition 12.23 (Rendering Map). For a fixed modality k , let O_k denote the space of possible outputs of that modality — the set of all texts, audio tracks, or videos of bounded practical length that could occupy $M(k)$. A *rendering map* for modality k is a function $\rho_k : \mathcal{P} \rightarrow O_k$ sending a provenance history to the content it produces in that modality.

Any modality with outputs of practically bounded length or duration has $|O_k| < |\mathcal{P}|$: there are only finitely many texts under any fixed length bound, or finitely many videos under any fixed duration and resolution bound, while $\mathcal{P}$ admits chains of unbounded length. This condition holds for every modality this essay has discussed — prose at any depth, audio, video, comics — since none of them is permitted to grow without bound merely to keep pace with an arbitrarily long provenance history.

Theorem 12.24 (No Faithful Rendering). *For any modality k with $|O_k| < |\mathcal{P}|$, every rendering map ρ_k is non-injective: there exist distinct provenance histories $S_1 \neq S_2 \in \mathcal{P}$ with $\rho_k(S_1) = \rho_k(S_2)$.*

Proof. Immediate from the pigeonhole principle: a function from a countably infinite set to a strictly smaller set cannot be injective. $\square$

This theorem is not a limitation of the calculus developed in this essay; it is a limitation on any architecture whatsoever that renders bounded outputs from unboundedly long histories, source-first or otherwise. It is worth being precise about what it does and does not say, because it is easy to mistake it for a restatement of the conflation problem this essay opened with.

Definition 12.25 (Erasure). A rendering pipeline exhibits *erasure* if, after producing $M(k)$ for some modality k , the provenance S used to produce it is discarded and becomes unavailable to future rule applications.

Definition 12.26 (Illusion). A rendering pipeline exhibits *illusion* with respect to modality k if a reader who observes only $M(k)$ cannot, from $M(k)$ alone, determine which of the (necessarily many, by Theorem 12.24) provenance histories in $\rho_k^{-1}(M(k))$ actually produced it.

Proposition 12.27 (What Source-First Architecture Eliminates). *A source-first architecture in the sense of Section 2 eliminates erasure: S is retained regardless of which renderings have been produced from it, per Definition 12.1. It does not eliminate illusion: by Theorem 12.24, any single rendering $M(k)$ remains the image of more than one possible provenance history, and observing $M(k)$ alone never distinguishes among them.*

Proof. Retention of S is immediate from Definition 12.1, in which S is a component of σ that persists across rule applications rather than being consumed by them; nothing in Definition 12.3 removes S upon producing a new populated modality. Non-elimination of illusion follows directly from Theorem 12.24: since ρ_k is non-injective, observing $\rho_k(S)$ alone cannot recover S uniquely, and this holds independently of whether S happens to be retained elsewhere in the system. $\square$

This is the essay's most honest statement of its own scope. Section 1 diagnosed a real and correctable pathology — treating a rendering as though it were the source, so that the source itself becomes unavailable for correction, branching, or citation. That pathology is erasure, and Proposition 12.27 shows the source-first architecture developed here genuinely eliminates it. But no architecture eliminates illusion, because illusion is a cardinality fact about the relationship between unboundedly long histories and boundedly sized outputs, not an engineering shortfall correctable by better design. A reader handed a short summary can never, from the

summary alone, rule out every other provenance history that would have produced the identical summary. What a source-first architecture changes is not whether this is true — it is always true — but what a reader can do about it.

Proposition 12.28 (Disambiguation by Depth). *Let $M(k_{\text{short}})$ and $M(k'_{\text{short}})$ be two short-depth renderings, in the sense of Section 3, produced from distinct provenance histories $S_1 \neq S_2$ with $\rho_{k_{\text{short}}}(S_1) = \rho_{k_{\text{short}}}(S_2)$ — an instance of illusion at that depth. If both S_1 and S_2 remain retained, then there exists a full-depth modality k_{full} such that $\rho_{k_{\text{full}}}(S_1) \neq \rho_{k_{\text{full}}}(S_2)$, and this rendering is recoverable by applying Q to the retained source.*

Proof. Take k_{full} to be a modality whose rendering map is the identity on $\mathcal{P}$ restricted to a faithful serialization of S , which exists for any retained S since nothing in Definition 12.1 bounds the length of a full-depth textual rendering the way this essay’s Theorem 12.24 bounds practically-sized modalities such as short summaries or fixed-duration video. Since $S_1 \neq S_2$, their full-depth renderings under this map differ. Recoverability follows from Definition 12.7: since S_1 and S_2 remain in dom, closure can be reapplied to either to populate k_{full} on demand. $\square$

This is the precise sense in which retaining provenance is not merely a courtesy to future editors but the mechanism by which illusion, though never eliminated outright, is made *reversible in principle* rather than permanent. A system that erases S after rendering has no such recourse: once S_1 and S_2 have both collapsed to the same short summary and the sources are gone, no amount of staring at the summary recovers which one actually occurred. A system that retains S can always, on request, regenerate a finer rendering and learn something Theorem 12.24 guarantees the coarser one could never have shown. What this essay’s architecture offers is not a way around the limit — there is none — but a guarantee that the limit is never made worse than it has to be.

This last point sharpens into a general fact about policies, not just about the two histories S_1, S_2 used above to illustrate it.

Definition 12.29 (Retentive and Erasing Policies). A rendering policy for σ is *retentive* if it preserves S indefinitely across every rule application, as in Definition 12.1. A policy is *k-erasing* if, after populating some modality $k \in T$, it discards S while keeping M , so that no rule chain terminating before the discard remains reconstructible.

Proposition 12.30 (Retention Dominates Erasure). *Fix σ and consider any finite sequence of future disambiguation requests, each of the form “recover a full-depth rendering distinguishing the histories that produced a given illusory output.” A retentive policy answers every such request, by Proposition 12.28. A k-erasing policy answers none of them for outputs populated at or before the discard, since the histories needed to distinguish them are no longer available to any rule chain. Retention therefore weakly dominates erasure on this class of requests: it never answers fewer of them, and it answers strictly more whenever at least one such request arrives after the point an erasing policy would have discarded S . Since nothing in Definition 12.1 or Definition 12.3 attaches a cost to retaining S — the entropy guard bounds the cost of applying rules, not of storing provenance, and Section 12.9 shows the whole calculus is satisfiable at zero cost throughout — this dominance is not offset by any corresponding advantage erasure holds elsewhere in the formalism.*

Proof. Immediate from Proposition 12.28, which requires only that S_1, S_2 remain in dom at the time of the request; a retentive policy satisfies this by definition for every request, however late it arrives. A k -erasing policy fails this requirement for any request arriving after the discard, since the provenance graph that Definition 12.7’s closure operator would need to reapply Q no longer exists in σ . The two policies therefore agree exactly on requests arriving before any discard and diverge on all requests arriving after, giving weak dominance with strict inequality whenever the latter set is nonempty. That retention carries no offsetting cost follows from Definition 12.1,

in which S is a component of σ read by the closure and disambiguation machinery but never consumed or bounded by the entropy guard of Definition 12.3. $\square$

What this establishes is narrower than a claim that history is valuable for its own sake, and correspondingly more useful: within this calculus, retention is never a worse policy than erasure on any measure the calculus itself defines, and is strictly better on exactly the measure — future disambiguation — that Theorem 12.24 shows can never be guaranteed in advance. The architecture does not need to argue that provenance matters as a philosophical primitive; it only needs Proposition 12.30, which shows that once illusion is conceded to be unavoidable, retaining the means to resolve any particular instance of it on demand is a dominant strategy rather than a mere preference.

The dominance is stated, and holds, over exactly the costs Definition 12.3’s entropy guard prices: the cost of applying a rule. It says nothing about storage, indexing, search, or the operational overhead of keeping a growing provenance graph available in a real deployment, because the calculus does not model those costs any more than it models the currency of an incentive layer. Section 12.9 draws this same boundary for a different reason — separating admissibility, closure, and coherence from any scheme that allocates resources or rewards contribution on top of them — and Proposition 12.30 inherits it rather than needing a separate argument for it. A deployment that finds storage or search costs prohibitive is not a counterexample to the proposition; it is a case where a resource-allocation layer external to the calculus, of exactly the kind Section 12.9 already flags as a separate axiom system, has to decide how much of the dominant policy it can afford to run.

12.9 The Incentive Layer Is a Separate Design Choice

Nothing in the preceding seven subsections requires attaching a cost, stake, or reward to any operation. Well-typedness, closure, entropy bounding, merge, coherence, and the rendering limits above are all satisfiable by a trivial system in which every rule has budget $\epsilon_r = 0$ and no resource is ever scarce; the proofs above go through unchanged in that degenerate case. It follows that any scheme for allocating attention or rewarding contribution — a market in stakeable credits, a reputation score, a bonding-and-slashing mechanism — is logically independent of the substrate developed here. Such a scheme may be layered on top for reasons of spam resistance or contributor incentive, but it is a separate axiom system, not a consequence of admissibility, closure, or coherence. A reader evaluating this architecture should keep the two questions apart: does the substrate guarantee that renderings stay faithful to their source, and separately, does any incentive layer built on top of it allocate resources in a way that is itself fair, contestable, and free of the attention-optimizing pathologies this essay set out to escape in the first place. The proofs above answer only the first question.

12.10 The Choke Point Fallacy

Section 12.9 separated the substrate’s guarantees from any scheme built on top of it for allocating resources. This subsection argues that one common such scheme — capping ingestion itself, the act of adding a new σ to the system at all — conflates two costs that the calculus keeps distinct, and that the conflation is not incidental but does real economic work for whoever imposes it.

Nothing in Definition 12.1 or Definition 12.3 attaches a resource cost to admitting an object. Well-typedness is a predicate on M and T ; it does not require σ to be replicated, indexed, served at low latency, or rendered on demand. A system that refuses ingestion above some fixed size is not protecting the substrate from an unaffordable operation — admission is definitionally free in the calculus — it is silently bundling four operations that a companion economic architecture

keeps separate: acceptance, preservation, availability, and discoverability. Preservation, availability, and discoverability have real marginal costs, paid in storage, bandwidth, and compute. Acceptance does not.

Definition 12.31 (Admission Policy). An admission policy for a corpus C is a predicate $\text{Admit}(\sigma)$ determining whether a proposed σ may be added to C . An admission policy is *size-gated* if $\text{Admit}(\sigma) = \text{false}$ whenever $|\sigma|$ exceeds some fixed bound L , independent of any other property of σ .

A size-gated admission policy is not a statement about resource constraints; it is a statement about which resource constraints the operator of C has chosen to make the contributor’s problem rather than the operator’s. The marginal cost of storing one additional object of size $|\sigma|$ does not disappear because the operator declines to store it — it simply never gets paid by anyone, which is a different design decision than the one a size cap advertises itself as making. The honest framing of a size-gated policy is therefore not “the system cannot afford this” but “the system has decided who bears the cost of affording this,” and the two framings license very different critiques.

The alternative this specification adopts: acceptance is unconditional — any well-typed σ is admitted to C and its provenance is retained, per Definition 12.1 and Proposition 12.27 — and the genuinely scarce resources, physical realization chief among them, are where cost is recovered. This does not make storage and bandwidth free; it makes them a separate accounting problem from admission, to be solved by whatever resource-allocation layer Section 12.9 already flagged as external to the calculus proper — not smuggled into the admission predicate where a contributor cannot see or contest it.

12.10.1 Compression Contribution as a Candidate Incentive Rule

Section 12.9 showed that an incentive layer is logically independent of the substrate and declined to specify one. This subsection specifies one candidate rule anyway, not because the substrate requires it but because Definition 12.9 already supplies the quantity a defensible rule would need, and leaving it unstated invites exactly the kind of ungrounded incentive scheme Section 12.9 warned against.

Definition 12.32 (Marginal Compression Contribution). For a corpus C and a candidate object σ_{new} , define

$$\Delta E(\sigma_{\text{new}} \mid C) = E(C \cup \{\sigma_{\text{new}}\}) - E(C),$$

using the entropy measure of Definition 12.9 extended additively over a corpus. A contribution is *compressive* if $\Delta E(\sigma_{\text{new}} \mid C) < 0$: adding σ_{new} lowers the corpus’s aggregate entropy, which by Definition 12.9 occurs exactly when σ_{new} ’s mutual information with existing content exceeds its own marginal H .

Proposition 12.33 (Compression Contribution Inherits the Measurement Problem). $\Delta E(\sigma_{\text{new}} \mid C)$ is exactly computable when C and σ_{new} are symbolic, in the sense of Section 13.2, and is available only as a learned, gameable proxy otherwise.

Proof. Immediate from Section 13.2: E itself has exactly this status, and ΔE is a difference of two evaluations of E , which cannot be more tractable than E is. $\square$

The honest scope of this proposal is therefore narrower than “pay contributors whose data compresses the system,” stated without qualification. It is: for symbolic and structurally-typed contributions — the event-graph fragments of Appendix A, structured metadata, deduplicatable content — ΔE is a real, auditable, non-gameable quantity a contributor can be paid against, and paying against it directly rewards the thing that actually reduces system-wide storage and retrieval cost, rather than rewarding volume of upload. For cross-modal contributions, any

compensation scheme built on ΔE inherits every caveat Section 13.2 already raised about proxy metrics and reward hacking, and should be deployed, if at all, with that inheritance disclosed rather than hidden behind a single reassuring number.

Two further obligations follow from treating this as a citable event rather than a silent ledger entry, per the reader-rights framework of Section 9. First, transparency: a contributor should be able to request $\Delta E(\sigma_{\text{new}} \mid C)$ for their own contribution at the time it was computed, not merely a resulting payment figure — the number, not just its consequence, is the auditable object. Second, the computation of ΔE is itself a rule application in the sense of Definition 12.3 and belongs in S : a payment made on the basis of a compression-contribution score that is not itself recorded in the corpus’s provenance graph is an unmarked rule in the sense of Definition 13.3, and Proposition 13.4 already showed why that is a problem worth avoiding rather than an implementation detail.

13 Three Problems Made Concrete

Section 12 establishes what the substrate guarantees given certain things are already in hand. Three of those things were asserted rather than shown: that prose admits an event grammar, that the entropy measure E is computable, and that the governance layer can determine canonicity without simply relocating the problem this essay set out to solve. Each is addressed directly below, and in each case the honest answer is more limited than the earlier sections implied.

13.1 A Worked Example of Prose Events

Take a short passage of argumentative prose: *“A teacher tries to increase a student’s capacity to evaluate multiple possibilities. A preacher tries to secure commitment to a single viewpoint. Real people usually fall somewhere between the two extremes.”* This is exactly the kind of definitional, distinction-drawing prose Section 4 worried a spatial event grammar could not capture. A bounded fragment of it can be captured as follows, using a small closed vocabulary of operations — `introduce_term`, `assert_relation`, `contrast`, `qualify` — each an event in the sense of Definition 12.3, with dependencies recorded explicitly rather than left to be inferred from sentence order:

```
event { id: e1, op: introduce_term, term: "teacher" }
event { id: e2, op: introduce_term, term: "preacher" }
event { id: e3, op: assert_relation, subject: e1,
        relation: "increases", object: "capacity to evaluate
        multiple possibilities", depends_on: [e1] }
event { id: e4, op: assert_relation, subject: e2,
        relation: "secures", object: "commitment to a single
        viewpoint", depends_on: [e2] }
event { id: e5, op: contrast, terms: [e3, e4], depends_on: [e3, e4] }
```

Prose is a rule from this event sequence to `PROSE-FULL`, exactly as Definition 12.3 describes: the passage above is recoverable by walking e_1 through e_5 in dependency order and rendering each operation through a fixed template. The interesting case is revision. Suppose the passage is later qualified: *“Most educators do some preaching, and most preachers do some teaching.”* This is a new event, not an edit to the old ones:

```
event { id: e6, op: qualify, target: e5,
        content: "most educators do some preaching, and most
        preachers do some teaching", depends_on: [e5] }
```

Events e_1 through e_5 are untouched. A diff against this source is the single added node e_6 and its edge into e_5 , not a line-level text diff against the rendered paragraph — which is exactly the property Section 4 wanted and Section 12’s provenance graph S was built to support. A renderer that has already cached the prose for e_1 – e_5 does not need to re-render them; only the portion of the output downstream of e_6 needs recomputation.

This demonstrates that a bounded fragment of argumentative prose — definitions, asserted relations, contrasts, qualifications — has a workable event grammar. It does not demonstrate that arbitrary theoretical prose does. Analogy, hedged claims, rhetorical questions, narrative exposition, and long inferential chains with implicit rather than declared dependencies are not covered by the four operations above, and it is not obvious that any small closed vocabulary of operations could cover them without either growing without bound or losing the very property — machine-precision — that motivated the event grammar in the first place. What is shown here is a genuine existence proof for a restricted case, not a general solution.

It is worth being precise about which claim this limitation actually threatens. Nothing in Definition 12.1 requires M to be populated entirely by fully eventized content, and nothing in Definition 12.3 requires every rule to originate from a maximally structured source. A semantic object is well-typed as long as $\text{dom}(M) \subseteq T$, regardless of how much internal structure the content at each modality carries; a source can therefore contain fully eventized passages, partially structured prose annotated with some but not all of the dependencies above, and raw unstructured prose side by side, coexisting in the same provenance graph. Read this way, the operative question is not whether the grammar of Section A can eventually cover arbitrary discourse — a demand this essay has no basis for meeting — but how much of a given source is worth eventizing at all, which is a cost-benefit question about where machine-precision earns its keep rather than a completeness question about the grammar’s expressive reach. The essay’s own claims should be read against the weaker requirement, since that is the one Sections 12 through 17 actually depend on: the calculus needs the canonical layer to be *more* structured than the rendering layer at whatever points structure has been supplied, not structured everywhere.

13.2 The Status of the Entropy Measure

Theorem 12.10 is unconditionally valid as mathematics: nothing in its proof depends on how E is computed, only on E being some well-defined real-valued function satisfying the stated guard. The open question is not whether the theorem is true but whether E , as defined in Definition 12.9, can actually be evaluated on real content.

For the worked example above, it can. The events e_1 through e_6 are symbolic objects drawn from a small, fixed operation vocabulary; H over a rule chain can be taken as the Shannon entropy of the empirical distribution of operation types in S , and MI between two symbolic fields — say, the set of terms introduced versus the set of terms qualified — is a standard, tractable computation over finite discrete distributions [1]. Nothing exotic is required.

For genuinely cross-modal content — a video rendering measured against the prose it was generated from — no such procedure is known. There is no principled, agreed definition of the Shannon entropy of a video clip or the mutual information between a video and a paragraph; any number produced for such a pair is, at present, necessarily a proxy computed by some learned multimodal model’s embedding similarity, not an instantiation of the formal quantity in Definition 12.9. This matters beyond mere imprecision. A proxy metric is optimizable, and any renderer whose acceptance depends on scoring low E against a learned proxy has an incentive to produce output that games the proxy rather than remains faithful to the source — the standard reward-hacking failure mode for any learned evaluation function substituted for

a formal one [2]. Section 12’s entropy-boundedness theorem does not prevent this, because the theorem’s guarantee is only as strong as the guard $E' \leq E + \epsilon_r$ actually being checked against a faithful E , and a gameable proxy is by construction not faithful in the cases that matter most. The honest position is that Section 12 gives an entropy accounting scheme that is rigorous for symbolic and single-modality content today, and that extending it to cross-modal content is a measurement problem, not a mathematics problem, requiring either a genuine theory of cross-modal information content or an explicit, independently audited proxy whose failure modes are tracked rather than assumed away.

The preceding two paragraphs are, in effect, describing two different quantities that Definition 12.9 lets a single symbol denote. Call them E_{symbolic} , the entropy of a finite discrete distribution over declared operation types, and E_{semantic} , whatever quantity would actually correspond to a reader’s sense of how much meaning drifted between a source and a distant rendering of it. Theorem 12.10 is a theorem about E_{symbolic} ; it is exact, computable, and exactly as rigorous as the worked example above suggests. The essay’s informal prose about “bounding semantic drift,” by contrast, is implicitly a claim about E_{semantic} , and no result in this essay establishes that E_{symbolic} tracks E_{semantic} once a rendering leaves symbolic content behind. Keeping the two notations distinct rather than letting one symbol carry both meanings is a discipline the essay owes itself: the theorem is not weakened by making this explicit, but its scope is narrowed to exactly where the proof actually reaches.

13.3 Canonicity, Defaults, and the Limits of the Substrate

The formal substrate of Section 12 has nothing to say about which branch a new reader is shown by default, and this is not an oversight to be patched but a boundary that should be stated precisely. Two different properties have been running together under the word “canonical,” and they need to be separated.

The first is a substrate guarantee: a branch, once created, is never deleted. It remains addressable by its identity I , its provenance S is preserved, and any rule chain that once applied to it can still be applied and cited. This is provided directly by the definitions in Section 12; nothing in the merge or closure operators removes a losing branch, because “losing” is not a concept the substrate defines. This guarantee is real and is not nothing: it is precisely what YouTube-style demonetization, shadow-banning, or platform-level deletion can violate, and a source-first system of the kind proposed here cannot silently make a branch harder to find in the way a recommendation algorithm can.

The second is a governance choice: which branch, among several that remain equally addressable, is shown to a reader who has not specified a coordinate — the branch returned when someone asks for “the current version” without further qualification. This is naturally written as a pointer-selection function $P : \mathcal{B} \rightarrow \mathcal{B}$ over the branch set $\mathcal{B}$, with $P(\mathcal{B}) = b_{\text{default}}$ picking out whichever branch is currently shown by default; “canonical,” in this narrower sense, is shorthand for $P(\mathcal{B})$ at a given time, not a property intrinsic to the branch itself. Writing it this way is worth doing even though P is not derived from the formal substrate, because it converts “this branch is canonical” into “this pointer currently resolves to this branch,” and the second phrasing makes it obvious that the fact could be otherwise tomorrow without anything about the branch having changed. P itself is not, and cannot be, derived from the formal substrate, because the substrate treats all valid branches symmetrically by design; the choice of P is external to it. Whatever mechanism instantiates P — vote, stake-weighted preference, editorial designation, reputation score — is exactly as capturable by concentrated resources as any comparable mechanism in any other social system, and no result in Section 12 bears on this. A faction with more resources can in principle secure $P(\mathcal{B})$ for its own branch under most plausible instantiations of P , including ones that look procedurally fair. Reader attention, and the

default-branch position in particular, behaves in this respect like the common-pool resources studied at length in the institutional-economics literature: hard to exclude anyone from, but rival in practice once a single default is imposed on all readers [11]. This essay does not attempt to adapt that literature’s design principles for self-governing institutions to the present case, but the resemblance is worth flagging, since it suggests the default-branch problem may be less novel, and less unprecedented, than a purely technical framing would imply.

Definition 13.1 (Branch Visibility and Capture). Let $V(b) = \Pr[P(\mathcal{B}) = b]$ denote the exposure share of branch b under some pointer-selection process, treated probabilistically if P varies over time or context. Let $A(b)$ denote an independent admissibility or audit score for b , supplied from outside the calculus in the sense of Section 13.7, normalized to the same scale as V . Define *capture* as

$$\text{Cap}(b) = V(b) - A(b).$$

Capture, in this sense, occurs precisely when a branch’s visibility exceeds what its independent audit score would warrant: $\text{Cap}(b) > 0$. This definition does not resolve the governance question this section has already declined to resolve — it presupposes A , an external admissibility judgment, exactly as Section 13.7 presupposes True as external to the calculus — but it gives the phrase “auditable capture” below a concrete form: an audit is, under this definition, nothing more than computing $\text{Cap}(b)$ for a disputed branch and publishing the result as a citable event in the sense of Section 14. What the substrate contributes is not a way to keep $\text{Cap}(b)$ at zero, which is a governance achievement no formal definition can guarantee, but a way to make $\text{Cap}(b)$ computable and disputable rather than a matter of unverifiable impression. Section 17 returns to instantiate $A(b)$ concretely, once executable citations and salience weighting are available to ground it.

What the substrate can offer here is narrower than a solution, but not nothing: the pointer-selection process itself can be made a first-class, versioned object subject to the same provenance discipline as everything else. A change to which branch is currently the default is then a citable event with an identity and a timestamp, not a silent update, and the history of default-pointer changes is itself queryable — one can ask not only “what is currently canonical” but “who made it canonical, when, and under what stated selection rule.” This converts unaccountable capture into auditable capture. It does not prevent capture, and any claim that it does would be overreach. The central political question the earlier critique identified — how does the system prevent a well-resourced faction from burying competing branches under an ostensibly neutral default mechanism — is not answered by this essay, and it is not answered by the formal substrate developed in Section 12, because it is not the kind of question a type system can answer. It is a governance design problem, it belongs to the PlenumHub layer discussed in the next section, and it deserves treatment on its own terms rather than a deferral disguised as an answer.

13.4 The Citation-Joint Problem: An Unmarked Rule

The preceding three subsections examine gaps between what Section 12 proves and what it can currently deliver. This one examines a gap the formal apparatus has not yet been asked to close at all: nothing in the definitions and propositions of Section 12 prevents a rule application from going undeclared while still changing what a reader takes the source to license.

To state the gap precisely, it helps to name a distinction Proposition 13.4 below already relies on without saying so. The provenance graph S attached to σ is a record of transformations, but a record is not automatically identical to the history it purports to record.

Definition 13.2 (Faithful Provenance). Let S_{recorded} denote the provenance graph actually stored with σ , and let S_{actual} denote the true sequence of transformations that produced σ from its sources. S is *faithful* if $S_{\text{recorded}} = S_{\text{actual}}$, written $\text{Faithful}(S)$.

An *unmarked rule*, defined next, is simply the mechanism by which faithfulness fails: a transformation that occurs as part of S_{actual} but never enters S_{recorded} .

Definition 13.3 (Unmarked Rule). Let σ be a semantic object with provenance graph S . An *unmarked rule* is a transformation applied in the course of producing σ' from σ that alters M — by adding content, narrowing a claim, or extending a claim beyond what its cited modality supports — without a corresponding node being added to S_{recorded} .

Theorem 12.4 proves that every *declared* rule application preserves well-typedness. It says nothing about undeclared ones, because the calculus has no way to detect a transformation that was never entered into the provenance record in the first place. An unmarked rule is invisible to the type system not because the type system is weak, but because there is nothing in S_{recorded} for it to fail to preserve.

This is not a hypothetical failure mode. Consider a citation of the form: “Villalobos et al. forecast that public human-generated text may be exhausted for LLM training between 2026 and 2032; looped computation can be viewed as a data-efficiency mechanism responding to this constraint” [3]. The first clause is a legitimate rule application: source paper to claim, faithfully rendered, checkable against the cited text. The second clause is a *different* rule application — an inferential move connecting data-efficiency-in-training to efficiency-of-inference-time-refinement — introduced by the citing author, not licensed by the source, and carrying its own entropy cost in the sense of Definition 12.9: it introduces a claim not present in, and not straightforwardly implied by, $M(\text{source})$. Written as a single sentence with a single citation attached, the two clauses appear to the reader as one rule application inheriting one warrant. They are not. The phrase “can be viewed as” is doing the work of a second, unmarked rule spliced onto the first, and no entry in S_{recorded} distinguishes where the cited claim ends and the citing author’s inference begins. It is worth noting that unfaithfulness of this kind can arise in either of two distinguishable ways: from a transformation that occurred but was never logged at all, or from a transformation that was logged as though it were licensed by its input when in fact its output exceeds what that input supports. The essay does not develop a separate entailment calculus to distinguish these two cases formally, since doing so would risk growing a second formal subsystem inside what is otherwise a single, unified account of provenance; both are adequately captured, for the purposes here, as failures of $\text{Faithful}(S)$.

Proposition 13.4 (Unmarked Rules Defeat Provenance Without Defeating Type-Correctness). *A chain containing an unmarked rule can satisfy $\sigma_n \vdash \text{valid}$ under Corollary 12.5 while $\neg \text{Faithful}(S)$.*

Proof. Corollary 12.5 establishes well-typedness by induction over *declared* steps in R ; an unmarked rule is by definition absent from R and from S_{recorded} , so its presence or absence has no bearing on the induction. σ_n can therefore be well-typed — every populated modality drawn from the declared set T — while $S_{\text{recorded}} \neq S_{\text{actual}}$, which is exactly $\neg \text{Faithful}(S)$. Well-typedness is a claim about the shape of M and T ; faithfulness of S to the true derivation history is a separate claim, and Corollary 12.5 was never a proof of the latter. $\square$

This matters more than a narrow reading of the type system might suggest, because it identifies exactly where the burden of trust silently shifts. A reader who checks a well-typed σ_n against the calculus and finds no violation may reasonably conclude the source-to-claim chain is intact. Proposition 13.4 shows this conclusion is unwarranted whenever an unmarked rule is present: type-correctness and provenance-faithfulness can diverge, and the calculus as developed in Section 12 certifies only the former. Framed this way, the Citation-Joint Problem is not a one-off anomaly particular to academic citation; it is a special case of provenance drift, the general phenomenon of S_{recorded} silently diverging from S_{actual} , of which an uncredited inferential leap is simply the most legible instance.

The remedy is not a new theorem but a discipline the existing machinery already supports:

treat every inferential move attached to a citation, however small, as its own rule with its own identity in S , rather than letting it travel for free inside the node representing the citation itself. Concretely, the sentence above would be two rule applications, not one: $r_1 : \text{source} \rightsquigarrow \text{forecast-claim}$, faithfully bounded by what the source states, and $r_2 : \text{forecast-claim} \rightsquigarrow \text{design-hypothesis}$, an inference by the citing author, carrying its own declared entropy budget ϵ_{r_2} and answerable on its own terms rather than borrowing the credibility of r_1 . Splitting the two is not pedantry; it is the only way S can do the job Section 12 was built around, which is making clear, for any claim, exactly how far the chain from source to conclusion actually reaches — and where, if anywhere, a reader is being asked to take a step the source itself did not license.

The same explanatory strategy appears outside citation analysis entirely: separate a bundled explanation into its component causes, construct an intervention that removes one component while holding the other fixed, and reassess which component was actually responsible for the observed effect. Diffusion-model training research has used exactly this move to distinguish a data-augmentation effect from a self-supervision effect that had previously been reported as one bundled improvement [4]. The parallel is structural, not substantive — this essay has nothing to say about noise scheduling — but the underlying move, holding one cause fixed and removing the other to see what was actually carrying the effect, is the same discipline the unmarked-rule diagnosis above asks a reader to apply to any citation that bundles a sourced claim with an uncredited inference.

Proposition 13.4 shows that an unmarked rule is invisible to the type system. This can be stated as a standing limit on verification itself, independent of any particular chain in which an unmarked rule happens to occur.

Theorem 13.5 (No General Verifier for Faithfulness). *Let V be any procedure that takes as input only S_{recorded} and M and outputs a judgment about whether $\text{Faithful}(S)$ holds, in the sense of Definition 13.2. There is no such V that decides $\text{Faithful}(S)$ correctly in general, because S_{actual} is not among V 's inputs.*

Proof. By Definition 13.2, $\text{Faithful}(S)$ is defined as the equality $S_{\text{recorded}} = S_{\text{actual}}$. A procedure $V(S_{\text{recorded}}, M)$ has no access to S_{actual} by hypothesis. For any such V , consider two scenarios that produce identical inputs (S_{recorded}, M) to V : in the first, σ was produced by exactly the rule chain recorded in S_{recorded} , so $\text{Faithful}(S)$ holds; in the second, σ was produced by that same declared chain with one additional unmarked rule inserted, in the sense of Definition 13.3, chosen so that its effect on M is indistinguishable from ordinary variation within the declared rule's output range, so $\text{Faithful}(S)$ fails. Since V receives the same (S_{recorded}, M) in both scenarios, it must return the same judgment in both, and so is incorrect in at least one of them. $\square$

It does not follow from Theorem 13.5 that every declared rule is equally opaque to a reader who suspects one is missing; some rules leave more to check than others, and it is worth naming the distinction precisely rather than leaving it as an intuition.

Definition 13.6 (Verifiable Rule). A rule $r : a \rightsquigarrow b$ is *verifiable* if there exists a procedure, independent of whichever renderer applied r , that decides from $M(a)$ and $M(b)$ alone whether $M(b)$ is a value r could have produced from $M(a)$.

A rule computing a checksum, a rule extracting a fixed field, or the rule linking e_1 through e_5 to their rendered paragraph in the worked example of Section 13.1 are all verifiable in this sense: an auditor holding only the declared input and output, without trusting whoever ran the rule, can confirm or refute the claimed relation. A rule that calls an opaque generative model with no reconstructable decision procedure is not.

Proposition 13.7 (Verifiability Narrows, but Does Not Resolve, the Unmarked-Rule Problem). *If every rule in a chain R is verifiable in the sense of Definition 13.6, an independent auditor can confirm that every declared step in R is consistent with its rule specification, and can falsify a claimed chain by*

exhibiting a step inconsistent with its declared rule. This does not establish Faithful(S) in the sense of Definition 13.2: an unmarked rule inserted between two declared verifiable steps, altering M without adding a node to S_{recorded} , remains undetectable by the same verification procedure, since that procedure checks whether declared steps were executed correctly, not whether every step that actually occurred was declared.

Proof. The first claim follows directly from Definition 13.6: verifiability of r_i guarantees a procedure deciding, from $M(a_i)$ and $M(b_i)$ alone, whether r_i could have produced b_i from a_i ; applying this procedure to every declared $r_i \in R$ either confirms all of them or exhibits the first inconsistent one. The second claim follows from Proposition 13.4: an unmarked rule is by definition absent from S_{recorded} , so no verification procedure applied to the declared steps of R has an opportunity to check it, regardless of whether the declared steps surrounding it are individually verifiable. $\square$

The value of Proposition 13.7 is narrower than a detection mechanism for unmarked rules, and it should not be read as one. What it supplies is a way to falsify a specific claimed derivation — an auditor can show that a declared step could not have produced its output — without supplying any corresponding way to certify that a chain is complete. A provenance graph built entirely from verifiable rules is one where every recorded claim can be independently checked; it is not, on that basis alone, a graph guaranteed free of unrecorded ones. The two failure modes this essay has already distinguished, erasure and illusion, both concerned what happens to information the calculus already represents; verifiability adds a third axis orthogonal to both, describing whether a recorded step can be checked by someone other than whoever recorded it, which is a property about trust in declared content rather than about completeness of the record.

13.5 Confabulated Closure

The unmarked rule of the previous subsection presupposes that some real inference occurred and simply went unrecorded. There is a second, more severe failure mode in which nothing licenses the added content at all — not even an uncredited inference — and it is worth distinguishing from the first, because the remedy for one does not address the other.

Definition 13.8 (Confabulated Closure). A rendering exhibits *confabulated closure* when it presents a modality k as populated, $k \in \text{dom}(M)$, for a σ that is not renderer-complete with respect to k — that is, $k \notin \text{Reach}(\sigma)$ — and the content occupying k was invented to fill the gap rather than derived by any declared rule or left absent.

Definition 12.7 makes Q undefined on objects that are not renderer-complete; the correct behavior when no rule reaches a required modality is for that modality to remain unpopulated, or for the rendering process to say plainly that it could not be produced. Confabulated closure is what happens when a rendering pipeline is asked to populate a modality anyway and, lacking a declared rule, produces plausible-sounding content rather than an empty slot. Unlike an unmarked rule, there is no real transformation to reconstruct here even in principle: a reader who suspects an unmarked rule can in principle work out what inference was likely made and check it against the source, because something was actually inferred. A reader who encounters confabulated closure has nothing to reconstruct, because nothing was inferred; a plausible-sounding value was generated to satisfy the shape of the expected output; with no argument behind it, faulty or otherwise.

This is a real and not merely hypothetical risk for exactly the kind of automated rendering pipeline this essay's Section 7 anticipates. Consider a pipeline asked to compress this essay's own treatment of governance, Section 13.3 above, into a single cell of a comparison table contrasting a conflated publishing era against a source-first one. This essay's own position is that

coordination and canonicity are *not* solved by the substrate — that default-branch selection is a governance choice external to the calculus, capturable by concentrated resources, and that the most the substrate can offer is auditable capture rather than prevented capture. A table cell has no room for that qualification; it wants a single confident word. A pipeline under that pressure may produce something like “solved” or “symmetrical,” neither of which this essay asserts anywhere, and neither of which follows from Section 13.3 by any argument, sound or unsound. That is confabulated closure: the modality “one-word governance summary” was never in $\text{Reach}(\sigma)$ for a source that spends a full subsection insisting the question is unresolved, and the correct output was either no cell at all or a cell reading “contested; see full discussion,” not an invented resolution.

The practical consequence is that a source-first system needs a declared failure mode for renderers, not merely a declared success mode. Q being undefined on a non-renderer-complete σ is a mathematical fact from Definition 12.7; it becomes a safeguard only if every rendering pipeline built on top of the calculus is required to surface that undefined-ness to the reader rather than silently discharging it with invented content. A closure operator that can fail loudly is a safety property. A rendering pipeline that never admits it has failed, and instead fills every requested slot with something confident-sounding, defeats that property regardless of how carefully Q itself was specified.

13.6 Self-Application: Are the Coherence Conditions Derived or Declared?

Theorem 12.17 rests on two hypotheses — rule determinism, and shared derivation from the same closed object — and both are taken as premises, not derived from anything prior. Nothing in Section 12 grounds determinism of the rule set in something more basic, and nothing derives “same closed object” from anything beneath Definition 12.1’s own stipulation of what a semantic object is. A theorem is entitled to its hypotheses; the point is narrower than a defect in the proof. It is that the coherence condition itself — the thing licensing PlenumHub’s claim that locally reasonable interpretive acts will not silently contradict one another at scale — is exactly as declared as any other axiom here, not derived from some deeper admissibility geometry that justifies it.

This matters because Section 20 describes PlenumHub as offering governance “emerging from admissibility conditions rather than imposed by centralized moderation,” language that implies the conditions are discovered rather than authored. Held to the standard this essay applies elsewhere, that phrasing overclaims. Two earlier cases in this essay’s own development were criticized for exactly this move: a governance aggregator whose defining axioms did not uniquely force the aggregator actually used, closed only by an undefended further assumption; and a multi-axis critique gate with no argument that its axes were complete, independent, or necessary rather than merely curated. Theorem 12.17’s hypotheses are in the same position. “Rendering rules are deterministic” and “contexts derive from the same closed object” are conditions the architecture requires, not conditions it explains the necessity of. A system built on nondeterministic rules with a probabilistic coherence guarantee, or on some weaker notion of “sufficiently related” closed objects, is not ruled out by anything proven here — it is simply a different design, and nothing above shows the conditions actually chosen are the right ones rather than merely workable ones.

A third stipulation belongs on this list alongside the two named above, surfaced only by comparison against an independently-derived operating-system specification built on a structurally similar coherence result (Section 20). Theorem 12.17’s proof establishes agreement on overlaps by noting both contexts compute “the deterministic image of $M(k)$ under *the same rule*,” which is a stronger requirement than hypotheses (i) and (ii) as separately stated make explicit: neither rules out two contexts independently possessing *different* deterministic rules for the same

declared modality, which would satisfy (i) and (ii) while still producing disagreement. The theorem should therefore be read as also requiring (iii): for every modality k populated in more than one context, the rules governing k are semantically equivalent on the shared source — agreeing in output whenever both are evaluated against it — not necessarily implemented as one literal shared instance. A single shared rule instance is a sufficient, easily checkable way to satisfy (iii); it is not shown to be necessary, and this essay does not claim otherwise.

None of this weakens Theorem 12.17 as mathematics; it is correct given its hypotheses. What it weakens is any suggestion that PlenumHub has solved the question of which continuations are admissible, rather than built a calculus that behaves well once a particular notion of admissibility has already been assumed. The honest claim is the smaller one: these conditions are self-consistent, not that they are non-arbitrary. Closing that gap would require showing the conditions are forced by something prior to their own statement, not merely that they happen to work — the same demonstration Section 13.1 asked for and did not supply for prose event grammars, and one that does not currently exist here or elsewhere this essay draws on.

A comparable staging choice, made for unrelated engineering reasons, is visible in recent work compiling natural-language task specifications into lightweight neural artifacts. Program-as-Weights produces a parameter-efficient adapter from a specification in two declared stages rather than one: a first pass emits an intermediate pseudo-program, and a second pass reads the specification together with that pseudo-program to produce the adapter itself [5]. Nothing about the compilation target forces this split; a single end-to-end pass from specification to adapter is not excluded by the problem, any more than Theorem 12.17’s hypotheses are excluded by anything prior to their own statement. The staged design was chosen because a named intermediate type is easier to inspect, debug, and constrain than an opaque one-shot mapping — which is close to Definition 12.3’s own reason for typing a rule by its source and target before asking what it computes. The parallel is worth naming for the same reason as Section 13.4’s: it is evidence that declaring an intermediate structure, rather than deriving it from necessity, is an ordinary and often correct engineering discipline elsewhere too, not a peculiarity of this essay’s own unresolved axioms.

13.7 Provenance Is Not Truth

Proposition 13.4 showed that well-typedness does not imply faithful provenance. A reader who has followed that result carefully may already suspect a further gap one level up, and it is worth closing explicitly rather than leaving it implicit: faithful provenance does not imply that anything recorded is correct.

This calculus has no formal notion of truth, and it should not manufacture one to answer this question. Let *True* denote any predicate a reader applies to M for reasons entirely external to σ — correspondence to some state of affairs the calculus does not represent, agreement with an independent authority, subsequent empirical confirmation, or any other standard a reader brings from outside this essay. Definition 13.2 places no constraint on what *True* means and neither does this subsection; *True* is supplied by the reader, not by the calculus, in exactly the sense that Section 12.9 supplies its cost model from outside the substrate rather than deriving it from admissibility or closure.

Proposition 13.9 (Provenance Is Not Truth). *For any predicate *True* external to this calculus in the sense above, $\text{Faithful}(S)$ neither implies nor is implied by $\text{True}(M)$.*

Proof. $\text{Faithful}(S) \not\Rightarrow \text{True}(M)$: Definition 13.2 requires only $S_{\text{recorded}} = S_{\text{actual}}$, a claim about whether the stored history matches the derivation that produced it. Nothing in that equality constrains the content of any event in the history; a chain of rule applications can be exactly and faithfully recorded while every assertion it makes is false, since Definition 12.3 governs the

type of a transformation, not the correctness of what it asserts. A σ built entirely from faithfully recorded, well-typed, false `assert_relation` events, in the sense of Appendix A, witnesses $\text{Faithful}(S) \wedge \neg \text{True}(M)$.

$\text{True}(M) \not\Rightarrow \text{Faithful}(S)$: a renderer may produce content satisfying `True` by a route entirely different from the one recorded in S_{recorded} — by guessing correctly, by an unmarked rule in the sense of Definition 13.3 that happens to preserve correctness, or by any process uncorrelated with the declared derivation. Since `True` is external to σ by hypothesis, nothing about M satisfying it constrains whether S_{recorded} matches S_{actual} ; a σ with correct content and a silently substituted derivation witnesses $\text{True}(M) \wedge \neg \text{Faithful}(S)$. $\square$

The asymmetry this proposition guards against runs in both directions, and each direction forecloses a different overclaim. That $\text{Faithful}(S) \not\Rightarrow \text{True}(M)$ prevents provenance from being mistaken for correctness: a reader who confirms that a claim’s derivation is faithfully recorded has confirmed only that the record is honest about its own history, not that the history began from true premises or reasoned validly. That $\text{True}(M) \not\Rightarrow \text{Faithful}(S)$ prevents the reverse mistake, in which a correct-seeming output is taken as evidence that the provenance behind it can be trusted. This completes a pattern the essay has maintained since Section 12: well-typedness does not imply faithful provenance, per Proposition 13.4; retention does not imply elimination of illusion, per Proposition 12.27; and now, faithful provenance does not imply truth. Each result narrows what the layer below it was ever entitled to promise the layer above. The substrate guarantees that a claim can be traced. It has never guaranteed, and this subsection makes explicit that it cannot guarantee, that what is traced is correct.

13.8 What This Calculus Guarantees, Enables, and Leaves Open

The preceding six subsections have shown, one at a time, that a formal result proved in Section 12 depends on a condition Section 12 does not itself supply. The pattern is worth collecting once, plainly, rather than leaving it distributed.

Three things are guaranteed outright, for every σ and every rule set, with no further condition to satisfy. Theorem 12.4 shows well-typedness survives any valid rule chain. Proposition 12.8 shows closure is idempotent wherever it is defined. Proposition 12.27 shows erasure is eliminated by construction, since Definition 12.1 never permits a rule application to discard S .

Six things are guaranteed conditionally: true given a stated hypothesis, not true regardless of it. Theorem 12.10’s drift bound holds given that E is computable, which Section 13.2 shows only for symbolic content. Theorem 12.17’s gluing result holds given rule determinism, semantic equivalence of rules governing a shared modality, and shared closed-object derivation, per Section 13.6’s now three-part accounting of what is stipulated rather than derived. Proposition 12.28’s illusion-resolution holds given that the histories in question remain retained and a full-depth rendering rule exists for them, neither of which the calculus forces into existence. Proposition 12.30’s retention dominance holds given the calculus’s own cost model, which prices rule application but, as stated directly after the proposition, prices neither storage nor search. Proposition 13.7’s auditability result holds given that every rule in a chain is verifiable in the sense of Definition 13.6, which is a property of the rule set an auditor is given, not something the calculus guarantees any particular rule set has. And Definition 12.13’s Φ_I must be supplied for guarded merge to resolve a merged object’s identity at all — Definition 12.11 alone leaves this genuinely open, per Proposition 12.14, not merely underspecified.

Three things the calculus does not address, and has not claimed to. Which branch a reader is shown by default is external to it, per Section 13.3. Whether an inference attached to a citation was actually recorded is undetectable by it, per Proposition 13.4 — and Proposition 13.7 sharpens rather than closes this gap, since verifying that declared steps are individually sound is not

the same as certifying that no step is missing. Whether a rendering pipeline invents content for an unreachable modality rather than reporting failure is unconstrained by it, per Definition 13.8. Each of these three is a property of a rule set, a renderer, or a governance process built on top of the calculus, not a property the calculus’s own definitions fix.

A deployment receives the first three guarantees automatically. It earns the middle six only by supplying the conditions they name — a computable entropy measure, deterministic and semantically-shared renderers, retained provenance, an affordable storage budget, an independently checkable rule set, and an explicit merge-identity policy. It must solve the last three itself, by engineering discipline or by a governance layer external to Section 12 entirely, since no theorem in this essay reaches them.

One further result sits outside this three-way ledger because it is neither a guarantee delivered to a deployment nor a gap left for one to fill: Proposition 13.9 holds unconditionally, for every σ , and states a ceiling rather than a floor. No amount of faithful provenance, verifiable rules, or auditable governance substitutes for an independent check on whether what is being traced is true. The calculus guarantees traceability; it was never in a position to guarantee correctness, and Proposition 13.9 is what makes that limit a proven fact rather than an implicit assumption.

14 Executable Citations

Section 13.4 argued that citations often conceal the boundary between sourced material and authorial inference. The problem is not merely that readers fail to inspect citations. It is that conventional citations provide almost no mechanism for inspection beyond locating a referenced document and manually reconstructing the reasoning that connects it to the claim being supported.

A source-first architecture permits a different conception of citation. Rather than functioning as a pointer to a document, a citation becomes a pointer to a provenance path. The cited object is not merely a location but a sequence of transformations connecting source material to a rendered claim — concretely, a chain of rule applications in the sense of Definition 12.3, each with its own type signature and entropy budget, rather than a single unstructured reference.

Suppose a sentence cites a research paper. Under ordinary publishing conventions, the reader receives a reference and little else. The intermediate steps connecting the paper to the author’s conclusion remain implicit. In a provenance-aware system those intermediate steps can be represented explicitly as rule applications within the graph, exactly as Section 13.4 proposed splitting a single bundled citation into r_1 and r_2 . The citation therefore resolves not only to a source but to a derivation.

This changes the meaning of verification. To verify a citation is no longer merely to check whether a document exists. It is to inspect the transformations connecting source and conclusion. A reader may accept the source while rejecting a particular inferential step, or may agree with an inference while disputing a later extrapolation. The citation itself exposes the structure required to make those distinctions.

This is close in spirit to the nanopublication proposal in the scholarly-communication literature, which represents individual scientific assertions together with explicit, machine-actionable provenance metadata rather than leaving provenance implicit in surrounding prose [10]; the difference here is one of scope rather than kind, since this essay’s executable citation exposes an entire derivation chain, in the sense of Definition 12.3, rather than attaching provenance to a single isolated assertion. What the preceding paragraphs have not yet done is say what an executable citation actually *is*, precisely enough to be checked rather than merely described. That is the burden of this section.

14.1 The Derivation, Not the Command

Definition 14.1 (Rule Version). A rule $r : a \rightsquigarrow b$ (Definition 12.3) may itself be revised over time — a renderer updated, a model retrained, an algorithm patched — without its type signature $a \rightsquigarrow b$ or entropy budget ϵ_r changing in the abstract. An immutable rule version is a tuple

$$r@v_r = (\text{hash}(r), a \rightsquigarrow b, \epsilon_{r,v}, \text{deps}_{r,v})$$

— a content-addressed identity $\text{hash}(r)$, the type signature, a version-specific entropy budget $\epsilon_{r,v}$, and a dependency closure $\text{deps}_{r,v}$ naming everything required to execute this version. $\epsilon_{r,v}$ is not assumed shared across implementations of the same signature: two implementations may satisfy the same $a \rightsquigarrow b$ while discarding different information, and are therefore different versions with potentially different budgets even when their declared types coincide. A human-readable label may accompany $\text{hash}(r)$ for legibility; it is not itself the immutability mechanism. This is an extension to Definition 12.3, not implied by it: nothing prior to this section required rules themselves to be versioned objects.

Definition 14.2 (Executable Citation). An executable citation is a tuple

$$\chi = (x_0, \delta_\chi, \text{VerifSpec}_\chi, \text{Ver}@v_{\text{ver}})$$

where $x_0 = I_{\text{src}}@v_{\text{src}}$ is the cited source’s immutable identity together with a fixed node in its provenance graph S_{src} ;

$$\delta_\chi = (r_1@v_1, \theta_1; \dots; r_n@v_n, \theta_n)$$

is a typed derivation path with intermediate objects $x_1, \dots, x_n$ and judgments $r_k@v_k : x_{k-1} \rightsquigarrow x_k$ for each k , where θ_k is the parameter and environment binding required to execute $r_k@v_k$ deterministically; VerifSpec_χ is a verification specification (Definition 14.3); and $\text{Ver}@v_{\text{ver}}$ is a verifier payload realizing it.

A single-rule citation is the degenerate case $n = 1$; nothing above requires it, and the introductory argument’s claim that a citation exposes “an entire derivation chain” is now the general case rather than an aspiration the formal definition failed to reach.

Definition 14.3 (Verification Specification). $\text{VerifSpec}_\chi = (P_{\text{claim}}, P_{\neg\text{claim}})$, a pair of predicates over the type of x_n , well-formed only if $P_{\text{claim}}(x) \wedge P_{\neg\text{claim}}(x) \Rightarrow \perp$ for all x — the two predicates may both fail to hold, but may never both hold.

Definition 14.4 (Verifier Payload). $\text{Ver}@v_{\text{ver}}$ is an implementation that, given x_n , reports which of $P_{\text{claim}}(x_n)$, $P_{\neg\text{claim}}(x_n)$, or neither holds. Like $r@v_r$, it is versioned and independently auditable, distinct from VerifSpec_χ itself. Placing a bare verification function inside χ would let a citation choose whichever verifier happens to declare it successful; separating the specification (what would count as support or contradiction) from the payload (whatever program actually checks it) keeps that choice auditable rather than folded invisibly into the citation’s own definition.

Definition 14.5 (Chain Execution). Given δ_χ and x_0 , define $x_k := r_k@v_k(x_{k-1})$ for $k = 1, \dots, n$ if deps_{r_k, v_k} are available and θ_k ’s environment is reproducible at each step. If any step fails, execution halts at the first failing k and χ is Inexecutable.

Definition 14.6 (Verification Result). $\mathcal{V} = \{\text{Verified}, \text{Contradicted}, \text{Undetermined}, \text{Inexecutable}\}$. For χ whose chain executes to x_n (Definition 14.5):

$$\text{Verify}(\chi) = \begin{cases} \text{Verified} & \text{if } \text{Ver}@v_{\text{ver}} \text{ reports } P_{\text{claim}}(x_n), \\ \text{Contradicted} & \text{if } \text{Ver}@v_{\text{ver}} \text{ reports } P_{\neg\text{claim}}(x_n), \\ \text{Undetermined} & \text{if } x_n \text{ is reached but neither is reported.} \end{cases}$$

If chain execution halts before x_n is reached, $\text{Verify}(\chi) = \text{Inexecutable}$.

Proposition 14.7 (Non-Verification Is Not Falsification). *Neither Undetermined nor Inexecutable implies claim is false, and neither licenses treating χ as though $\text{Verify}(\chi) = \text{Contradicted}$.*

Proof. By Definition 14.6, Contradicted requires $\text{Ver}@v_{\text{ver}}$ to report $P_{\neg\text{claim}}(x_n)$ specifically. Undetermined is exactly the case where x_n was reached but this report did not occur — execution succeeding is not the same event as contradiction being established, and Definition 14.3’s well-formedness condition prevents P_{claim} and $P_{\neg\text{claim}}$ from standing in for each other. Inexecutable is the case where x_n was never reached at all. The three outcomes are pairwise disjoint by construction, and no rule above infers one from another. $\square$

An analysis finding no significant effect is the paradigm case of Undetermined, not Contradicted: x_n is reached, $P_{\neg\text{claim}}(x_n)$ (a specific, positive disconfirming result) was not established merely because $P_{\text{claim}}(x_n)$ was not established either. Collapsing Undetermined into Contradicted would silently convert every underpowered study into a refutation, which is precisely the failure mode this four-way result type exists to block.

This distinction is not a technicality more broadly, either. A citation whose source has gone offline, whose interpreter version is no longer installable, or whose environment cannot be reconstructed is Inexecutable for reasons entirely about *availability* — whether x_0 and a compatible interpreter for δ_χ can currently be retrieved — which is a separate question from *auditability*: whether $\text{Verify}(\chi)$ is well-defined in principle, given availability. A citation can be fully auditable and presently inexecutable at once; treating the latter as evidence against the former is precisely the error Proposition 14.7 rules out. This essay does not develop a full availability/auditability calculus here — that is future work — but the distinction is load-bearing enough to state now rather than let Inexecutable quietly collapse into Contradicted in practice.

Proposition 14.8 (Verifier Trustworthiness Requires Verifiability). *$\text{Verify}(\chi)$ is trustworthy to a third-party auditor only if $\text{Ver}@v_{\text{ver}}$ is itself a verifiable rule in the sense of Definition 13.6, and each $r_k@v_k \in \delta_\chi$ is independently verifiable.*

Proof. If $\text{Ver}@v_{\text{ver}}$ is opaque, an auditor examining Verified or Contradicted has no way to confirm the report without trusting whoever ran it, the deference Section 15 distinguishes from auditing. Separately, Proposition 13.7 already established that confirming each declared $r_k@v_k$ is individually verifiable lets an auditor check that each declared step could have produced its output, but does not, per Proposition 13.4, certify that no unmarked rule was spliced between x_{k-1} and x_k . Definition 14.2’s explicit chain gives every step a place to be checked, reducing the opportunity for such splicing to go unnoticed; it does not repeal Theorem 13.5’s standing limitation. $\square$

14.2 Citation as Typed Relation, Not Payload

Definition 14.9 (Citation Derivation). The semantic content of χ is the pair $(\delta_\chi, \text{VerifSpec}_\chi)$: the typed chain from x_0 to x_n , and the specification x_n is judged against. Any shell command, notebook, or script realizing δ_χ , and any program realizing $\text{Ver}@v_{\text{ver}}$, are payloads implementing this content, not the content itself. Two citations sharing $(\delta_\chi, \text{VerifSpec}_\chi)$ but implemented by different payloads are the same citation for auditing purposes; a payload’s mere runnability, absent a VerifSpec_χ binding it to a specific claim, is reproducibility, not citation.

This is the precise sense in which an executable citation differs from an ordinary link and from merely reproducible research. A link supplies no δ_χ or VerifSpec_χ at all. Reproducible research supplies a runnable δ_χ but need not bind it to any VerifSpec_χ — it is checkable by anyone willing to run it, but asserts nothing about which sentence in a citing text it verifies. An executable citation requires that binding, typed and checkable, which is why Definition 14.9 states $(\delta_\chi, \text{VerifSpec}_\chi)$ as the citation’s content and treats every payload beneath it as replaceable.

Chain composition also inherits an existing result for free: by Theorem 12.10, the total entropy cost of δ_χ is bounded by $\sum_{k=1}^n \epsilon_{r_k, v_k}$, so a citation’s derivation carries an explicit, auditable drift budget rather than an unstated one accumulated silently across n uninspected steps.

14.3 Preservation and the Problem of “Latest”

Proposition 14.10 (Historical Validity Survives Rendering Change). *If σ_{src} ’s default pointer $P(\mathcal{B})$ (Section 13.3) later resolves to a different branch, χ ’s denotational validity — that δ_χ refers to the specific $x_0, \dots, x_n$ it was authored against — is unaffected, provided v_{src} remains addressable within S_{src} . This does not by itself guarantee χ remains executable.*

Proof. By Lemma 19.1, I_{src} ’s immutability guarantees v_{src} remains uniquely resolvable within S_{src} regardless of further revisions, since S_{src} is append-only (Proposition 12.27). δ_χ references x_0 and each $r_k@v_k$ by content identity ($\text{hash}(r)$, Definition 14.1) directly, not through $P(\mathcal{B})$, so a change to which branch is shown by default changes what a reader sees when no coordinate is specified, not what v_{src} or any $\text{hash}(r_k)$ denotes when addressed explicitly. This establishes only that χ continues to denote a fixed derivation; whether that derivation can still be *executed* — whether x_0 ’s content and each $r_k@v_k$ ’s interpreter remain retrievable — is governed separately by Definition 14.5 and is not guaranteed by append-only retention alone. $\square$

Remark 14.11 (Denotation Is Not Availability). Append-only retention and content-addressed identity guarantee χ continues to denote a fixed, well-defined derivation. They do not guarantee any interpreter for $r_k@v_k$ still exists, or that x_0 ’s content remains retrievable from wherever it was stored. A citation can therefore remain historically valid — fixed and unambiguous in what it denotes — while becoming Inexecutable as environments, interpreters, or storage decay. This is the precise content of the availability/auditability distinction raised earlier. “Historically valid” means denotationally fixed, a strictly weaker property than “currently executable,” and Proposition 14.10 is deliberately a claim about the former.

Proposition 14.12 (Unpinned “Latest” Is an Unmarked-Rule Vector). *A citation binding $x_0 = I_{\text{src}}@\text{latest}$, where $\text{latest}(\sigma_{\text{src}}, t)$ is evaluated at execution time t rather than pinned at authoring time t_0 , does not supply a well-defined δ_χ , and its later execution constitutes an unmarked rule in the sense of Definition 13.3.*

Proof. Definition 14.2 requires x_0 fixed; $\text{latest}(\sigma_{\text{src}}, t)$ varies with t , so δ_χ denotes a family of derivations rather than one. Executing χ at some $t' > t_0$ against $\text{latest}(\sigma_{\text{src}}, t')$ performs a resolution step affecting x_0 that was never entered into S_{recorded} at t_0 , satisfying Definition 13.3 directly. $\square$

The remedy is the same discipline Section 13.4 already recommended: resolve “latest” once, at t_0 , into a concrete v_{src} , and record that resolution as part of χ . A citation to “latest” without this pinning is an ordinary link wearing an executable citation’s vocabulary; Proposition 14.12 is the formal statement of why the two must not be confused.

15 The Reader as Auditor

Section 14 showed that a citation in this architecture resolves to a derivation rather than merely a document. This section draws out a consequence of that shift for the reader’s own role, since a system that changes what a citation is also changes what reading a citation means to do.

Conventional publishing asks a reader to trust an author’s chain of reasoning up to the point where a citation is offered, and then to trust the cited source in turn, without any practical

mechanism for inspecting the joints between the two. The posture this asks of a reader is fundamentally one of deference: verification is possible in principle, by locating the cited document and reconstructing the reasoning independently, but the architecture does nothing to make that reconstruction easy, and in practice most citations go unchecked for exactly this reason.

A source-first architecture, once executable citations and the provenance apparatus of Section 12 are in place, asks something different of a reader. Because every claim resolves to a chain of rule applications rather than an opaque reference, and because Definition 13.6 identifies which of those rule applications can be independently checked without trusting whoever applied them, a reader is positioned to inspect a derivation rather than merely accept or reject its conclusion. This is the posture of an auditor rather than a consumer: not trust extended by default and withdrawn only on suspicion, but verification treated as a normal and low-cost first step.

It is important not to overstate what this shift accomplishes. Proposition 13.4 already showed that an unmarked rule is invisible to any verification an auditor can perform, and Theorem 13.5 sharpens this into a standing limitation: no procedure restricted to S_{recorded} and M can certify faithfulness in general. An auditor-reader is therefore not guaranteed a complete account, only a more inspectable partial one. What changes is not the ceiling on what verification can achieve — Proposition 13.9 sets that ceiling regardless of architecture — but the floor: a reader who wants to inspect a derivation now has a mechanism for doing so that a conventional citation does not provide, even though that mechanism inherits every limitation the calculus has already been honest about.

The auditor posture also reframes what a reader owes a claim before rejecting it. Under a deference model, rejecting a claim without reading its source is a minor discourtesy. Under an auditor model, the derivation is available at whatever depth a reader is willing to inspect, in the sense of Section 12.7, which makes unexamined rejection and unexamined acceptance symmetric failures rather than asymmetric ones. Neither failure is prevented by the architecture; both become more visible as failures, because the alternative — actually looking — has been made cheaper than it was before.

16 The Cost of Forgetting

Section 12 treated erasure, in the sense of Definition 12.25, as a failure mode to be eliminated, and Proposition 12.27 showed the architecture developed there eliminates it by construction. That result is correct as far as it goes, but it invites a misreading worth heading off before the next section turns to the architecture's own costs: it does not follow that forgetting is always a defect, or that a system committed to retention has thereby avoided every cost associated with compression.

Every depth transformation in the sense of Section 3 is, definitionally, a form of forgetting: a short rendering discards information a full rendering retains, and Theorem 12.24 already established that this discarding is unavoidable for any practically bounded output. Compression of this kind is not the pathology Section 1 opened by diagnosing. The pathology was never that information gets compressed; it was that the compressed form gets mistaken for, or substituted for, the source it was compressed from, so that the discarded information becomes permanently unrecoverable rather than provisionally set aside.

This distinction gives forgetting two forms that this essay's vocabulary has kept separate without naming the separation explicitly. *Reversible forgetting* is any depth reduction performed on a retained source: the full history remains in S , per Definition 12.1, and Proposition 12.28 shows the discarded detail can always be recovered on request. *Irreversible forgetting* is erasure in the

sense of Definition 12.25: the source itself is discarded, and no amount of inspecting the remaining rendering recovers what was lost. The architecture developed in this essay does not, and cannot, eliminate the practice of compression — Section 11 depends on compression being available at will — but it converts every instance of forgetting from irreversible to reversible by construction, provided the retention discipline of Definition 12.29 is actually followed.

The remaining cost is therefore not the cost of compressing content, which this architecture treats as ordinary and necessary, but the cost of retaining what compression discards. That cost is real, is not priced by the entropy guard of Definition 12.3, and is exactly what the next section’s discussion of provenance overload takes up directly: reversibility is not free merely because it is not erasure, and a system that can never forget anything irreversibly may still find that the volume of what it reversibly retains becomes its own kind of burden.

17 Failure Modes of Source-First Systems

The preceding sections have emphasized the limitations of artifact-centered publishing and the advantages of treating structured sources as canonical. It would be a mistake, however, to conclude that source-first systems are free from failure modes of their own. Every architecture resolves some constraints only by introducing others.

The most obvious risk is provenance overload. A system that retains every transformation, branch, qualification, and merge may eventually present readers with more history than they can practically inspect. Preservation of information does not guarantee comprehension. A graph that faithfully records every decision can still become too large for human navigation.

Proposition 17.1 (Provenance Overload Bound). *Let $|S|$ denote the number of nodes in a provenance graph, and let B denote a reader’s practical inspection budget, measured in the same units. If $|S| > B$, full inspection of S by that reader is impossible. Define the inspectable fraction $\eta = B/|S|$. For fixed B , $\eta \rightarrow 0$ as $|S| \rightarrow \infty$.*

Proof. The first claim is immediate: a reader who can inspect at most B nodes cannot inspect all of $|S| > B$ nodes, since inspection is assumed here to consume budget at a rate of at least one unit per node. The second claim follows directly from the definition of η : for fixed B , $\lim_{|S| \rightarrow \infty} B/|S| = 0$. $\square$

This is a deliberately minimal formalization — it does not model that some nodes are more important to inspect than others, nor that inspection budget might scale with the significance of a dispute rather than remaining fixed — and it should not be read as more than what it is: a precise statement of the sentence already made in prose above, that preservation of information does not guarantee comprehension. Proposition 17.1 shows this is not merely likely but structural: for any fixed reader capacity, growth in $|S|$ drives the inspectable fraction to zero, regardless of how faithfully S is maintained.

17.1 Provenance Overload, Revisited: Salience-Weighted Inspection

Proposition 17.1 was explicit about its own limits: it does not model that some nodes matter more than others, nor that a reader’s effective budget might scale with what is actually in dispute. Executable Citations (Section 14) now supplies exactly the structure needed to remove both limitations — not by repealing the bound, which remains true as stated, but by showing that the quantity a reader auditing a specific claim actually needs to control is not $|S|$ at all.

Definition 17.2 (Structural Salience). For $e \in S$ produced by rule application $r@v$ (Definition 14.1), the salience of e is $w(e) := \epsilon_{r,v}$, its own declared entropy budget. For a chain $\delta = (r_1@v_1; \dots; r_n@v_n)$, the chain’s total salience is $W(\delta) = \sum_{k=1}^n w(e_k)$.

This is deliberately the structural notion, not a statistical one. Section 13.2 already distinguished E_{symbolic} , exact and computable, from E_{semantic} , a proxy at best for cross-modal content; defining $w(e)$ as a declared budget already present in S 's own construction keeps salience in the computable, symbolic category rather than smuggling in an uncomputed mutual-information estimate this essay has already flagged as unreliable outside that category.

Proposition 17.3 (Weighted Selection Dominates Uniform Sampling). *For fixed budget $B \leq |S|$, the B nodes of highest $w(e)$ capture aggregate salience $W_B \geq W(S')$ for any other size- B subset $S' \subseteq S$.*

Proof. Immediate from the definition of a top- B selection by a scalar weight: any subset achieving a strictly greater sum would contain some element with $w(e)$ exceeding the B -th largest weight, contradicting that the top- B set was chosen by that ordering. $\square$

Proposition 17.4 (Chain-Scoped Auditing Is Independent of $|S|$). *Verifying a single executable citation χ with $|\delta_\chi| = n$ requires inspection budget $B \geq n$, independent of $|S|$.*

Proof. By Definition 14.5, computing x_n from x_0 touches only $x_0, \dots, x_n$ and the rule applications $r_1@v_1, \dots, r_n@v_n$ constituting δ_χ . No other node of S , nor any node of any other object's provenance graph, is referenced by this computation or by $\text{Verify}(\chi)$ (Definition 14.6), which depends only on x_n and VerifSpec_χ . $\square$

Corollary 17.5 (Chain-Scoped Coverage Is Complete). *A reader with $B \geq n$ who inspects exactly δ_χ captures $W_B = W(\delta_\chi)$, the entirety of χ 's own declared salience, regardless of how large the ambient $|S|$ is.*

Proof. Immediate from Proposition 17.4: since verifying χ never requires any node outside δ_χ , a budget covering δ_χ exactly is a budget covering everything salient to χ , by Definition 17.2's restriction of w to nodes actually produced along a declared chain. $\square$

This resolves, rather than merely restates, the tension Proposition 17.1 left open. That proposition's pessimism is about a reader trying to understand an entire object — every branch, every qualification, every rendering choice — as $|S|$ grows without bound, and nothing here changes that conclusion for that task. But auditing one claim was never the same task as understanding the whole object, and Corollary 17.5 shows the two tasks have genuinely different, not merely differently-perceived, resource requirements: $\eta = B/|S| \rightarrow 0$ governs the first; $B \geq n$, with n typically orders of magnitude smaller than $|S|$ and entirely independent of it, governs the second. A reader disputing a specific claim naturally allocates B to that claim's δ_χ rather than uniformly across S — which is exactly the budget-scales-with-the-dispute behavior Proposition 17.1 flagged as unmodeled, now given a precise account rather than left as an acknowledged gap.

Proposition 17.6 (Weighting Does Not Rescue General Inspection). *For a reader whose task is not scoped to any single δ_χ — inspecting S generally, per Proposition 17.1's original setting — salience weighting improves which B nodes are chosen but does not change $\eta = B/|S| \rightarrow 0$ itself.*

Proof. η is defined purely as a ratio of cardinalities in Proposition 17.1 and does not reference w . Proposition 17.3 maximizes W_B for fixed B ; it does not increase B or decrease $|S|$, so η 's limit is unaffected by which B nodes are selected. $\square$

Proposition 17.6 is included to prevent Corollary 17.5 from being misread as a general fix. It is not one, and this section does not claim otherwise: salience weighting and chain-scoping repair the specific case Executable Citations was built for — auditing a claim — and leave the general overload problem exactly as pessimistic as Proposition 17.1 already showed it to be.

17.2 Instantiating $A(b)$: A Citation-Grounded Audit Score

Definition 13.1 left $A(b)$, the independent admissibility or audit score against which a branch's exposure share $V(b)$ is compared, "supplied from outside the calculus." That was honest at the time — nothing prior to Section 14 gave this essay a computable candidate for A . Executable citations now do, for exactly the portion of a branch's content that carries them, and this section states the construction precisely rather than merely noting that one is now possible.

Definition 17.7 (Verification Score). $s : \mathcal{V} \rightarrow [0, 1]$, $s(\text{Verified}) = 1$, $s(\text{Undetermined}) = 1/2$, $s(\text{Contradicted}) = 0$. s is undefined on Inexecutable.

Definition 17.8 (Citation-Grounded Audit Score). For a branch b whose content includes executable citations $\chi_1, \dots, \chi_m$, let $X(b) = \{\chi_i : \text{Verify}(\chi_i) \neq \text{Inexecutable}\}$. If $X(b) \neq \emptyset$,

$$A(b) := \frac{\sum_{\chi \in X(b)} W(\delta_\chi) \cdot s(\text{Verify}(\chi))}{\sum_{\chi \in X(b)} W(\delta_\chi)},$$

using $W(\delta_\chi)$ from Definition 17.2 as the weight. $A(b)$ is undefined if $X(b) = \emptyset$.

Proposition 17.9 (Inexecutable Citations Neither Inflate Nor Deflate $A(b)$). *$A(b)$ as defined is independent of how many citations in b are Inexecutable, and independent of any weight those citations would have carried had they executed.*

Proof. Immediate from Definition 17.8: both sums range only over $X(b)$, which excludes every χ with $\text{Verify}(\chi) = \text{Inexecutable}$ by construction. This is required by Proposition 14.7: since in-executability is not evidence of falsity, an audit score that penalized it would be manufacturing exactly the inference that proposition forbids. $\square$

Proposition 17.10 ($A(b)$ Is Computable When Its Citations Are Verifiable). *If every $\chi \in X(b)$ satisfies the hypothesis of Proposition 14.8 (each $r_k @ v_k \in \delta_\chi$ and $\text{Ver} @ v_{\text{ver}}$ independently verifiable), then $A(b)$ is not merely defined but independently checkable by a third-party auditor, rather than a number a publisher must be trusted to report.*

Proof. Direct composition: Proposition 14.8 gives an auditor grounds to trust each $\text{Verify}(\chi_i)$ individually; Definition 17.7 and Definition 17.8 are then a fixed, public arithmetic function of those trusted results and the already-public weights $W(\delta_{\chi_i})$ (Definition 17.2). No step requires deferring to an unaudited party. $\square$

This is a narrower claim than " $A(b)$ is now solved," and the narrowing is worth stating explicitly rather than left implicit. Two gaps remain, and neither is closed by the construction above.

Remark 17.11 (Coverage, Not Truth). $A(b)$ as defined says nothing about content in b carrying no executable citation at all — it measures the verification quality of what is cited, not the completeness of what should have been. A branch consisting entirely of confident, uncited assertion has $X(b) = \emptyset$ and hence undefined $A(b)$, which this construction correctly refuses to paper over with a default value; an undefined audit score is a more honest signal than a fabricated one. Nor does a high $A(b)$ certify that b 's claims are *true* in the sense of Section 13.7 — $A(b)$ aggregates Verify results, and Proposition 13.9 already established that faithful, checkable provenance neither implies nor is implied by truth external to this calculus. $A(b)$ is therefore an audit score in the narrow sense of measuring citation-verification discipline, not a truth score, and $\text{Cap}(b) = V(b) - A(b)$ should be read accordingly: a branch can show low capture by this measure while still being wrong about everything it correctly cited, and can show apparent capture while resting on claims no available citation could test.

Remark 17.12 (A Deeper Grounding Remains Open). The uncited case in Remark 17.11 is precisely the gap a general theory of admissibility — evaluating whether a claim belongs to the admissible continuations of some governing structure, independent of whether any citation

happens to test it — would need to address to supply $A(b)$ a value where Definition 17.8 currently supplies none. Ongoing work on admissibility as a temporally-stratified structure is a plausible source of such a grounding, but that work is not yet developed to the point of a plug-gable, scalar audit function, and this essay does not attempt to force a premature merger. $A(b)$ therefore remains partially instantiated: computable and auditable wherever executable citations exist, honestly undefined where they do not, pending whatever a mature admissibility theory eventually supplies for the latter case.

A second risk is branch proliferation. The architecture deliberately lowers the cost of creating alternative continuations. This encourages experimentation and protects minority viewpoints from deletion, but it also produces a growing population of partially developed branches competing for attention. The problem shifts from preserving alternatives to discovering which alternatives deserve examination.

A third risk concerns governance. Section 13.3 showed that no formal property of the substrate determines which branch should be shown by default. Any mechanism that resolves this question introduces the possibility of strategic influence. Source-first systems make such influence auditable, but auditability should not be mistaken for immunity.

A fourth risk is renderer monoculture. The architecture assumes a diversity of rendering pipelines capable of producing alternative views of the same source. In practice, users may converge on a small number of popular renderers. When this occurs, the system can recreate many of the interpretive bottlenecks associated with conventional media despite retaining a richer substrate underneath. A plurality of sources does not guarantee a plurality of perspectives if every reader encounters them through the same transformation process.

Proposition 17.13 (Renderer Concentration). *Let $\{r_1, \dots, r_m\}$ be the set of renderers available for some modality, and let $p(r_i)$ denote the empirical share of renderings produced by r_i across some population of readers, with $\sum_i p(r_i) = 1$. Define renderer concentration as*

$$C_R = \sum_i p(r_i)^2.$$

Then $C_R \rightarrow 1$ as usage concentrates on a single renderer, and $C_R = 1/m$ when usage is uniform across all m renderers.

Proof. If $p(r_1) = 1$ and $p(r_i) = 0$ for $i \neq 1$, then $C_R = 1^2 = 1$, the maximum value the sum can take given $\sum_i p(r_i) = 1$. If $p(r_i) = 1/m$ for every i , then $C_R = \sum_{i=1}^m (1/m)^2 = m \cdot (1/m^2) = 1/m$, which is the minimum value C_R can take given m available renderers, since a sum of squares subject to a fixed-sum constraint is minimized at the uniform point. $\square$

The substrate developed in Section 12 guarantees only that multiple renderers *may* exist for a given modality, per Definition 12.3; it says nothing about C_R , because usage concentration is a fact about reader behavior, not about the rule set. A high C_R is compatible with a well-typed, renderer-complete, fully coherent σ in the sense of Section 12, since none of those properties constrain which renderer a population of readers actually chooses. This is the precise sense in which the failure mode named above is a property of usage rather than of the calculus: a plurality of declared rules is necessary but not sufficient for a plurality of renderings actually reaching readers, and C_R is what would have to be measured, in a real deployment, to know whether the necessary condition has translated into the sufficient one.

17.3 Renderer Concentration, Strengthened: Corroboration Requires Diversity

Proposition 17.13 and its two limiting cases are, by themselves, little more than a Herfindahl index restated in this essay's notation. Executable citations give C_R somewhere real to bite: not

on the substrate’s guarantees, which Proposition 17.13 already correctly says are untouched by it, but on how much evidentiary weight a reader is entitled to place on multiple citations independently reporting Verified.

Definition 17.14 (Verifier Population Concentration). For a class of citations sharing a VerifSpec type, let $q(v_j)$ be the empirical share of Verify executions performed by verifier implementation $\text{Ver}@v_j$ (Definition 14.4) across some population of executions. Define

$$C_{\text{Ver}} = \sum_j q(v_j)^2,$$

exactly as Proposition 17.13 defines C_R , applied to verifiers rather than renderers.

Proposition 17.15 (Corroboration Requires Diversity). *If n citations each report $\text{Verify}(\chi_i) = \text{Verified}$, and $C_{\text{Ver}} = 1$ for the verifier population producing those n reports (a single $\text{Ver}@v$ performed all n executions), then the n reports carry at most the evidentiary weight of one independent check, not n .*

Proof. By Definition 14.4, $\text{Ver}@v$ is one fixed, deterministic implementation. Applying it to n distinct $(x_n, \text{VerifSpec})$ pairs tests whether v ’s judgment is internally consistent across those n cases, which is a fact about v ’s behavior, not independent corroboration of any individual result: a systematic error in v — a bug in how P_{claim} or $P_{\neg\text{claim}}$ is actually implemented, a biased tolerance, a mishandled edge case — would be reproduced identically across all n executions rather than surfaced by disagreement, since disagreement requires a second, differently-implemented v' to have been exercised on at least one of the n cases and to have produced a different result. $C_{\text{Ver}} = 1$ means no such v' was exercised anywhere in the population; hence the n reports are n observations of one implementation agreeing with itself, which is not what “ n independent verifications” ordinarily means. $\square$

Corollary 17.16 (Concentration Discounts $A(b)$ ’s Practical Independence). *Proposition 17.10 established that $A(b)$ is independently checkable by a third-party auditor whenever its constituent citations are formally verifiable (Definition 13.6). If the verifier population computing those citations’ Verify results has $C_{\text{Ver}} \rightarrow 1$, “independently checkable” remains true in the sense of Proposition 17.10 — an auditor could re-run the check with a different implementation — but $A(b)$ ’s constituent Verified results, as actually produced, do not yet constitute the corroboration that formal checkability promises, per Proposition 17.15.*

Proof. Immediate composition of the two propositions: Proposition 17.10’s claim is about the existence of an independent checking procedure, which C_{Ver} does not affect, since Definition 13.6 is a property of the rule, not of usage statistics. Proposition 17.15’s claim is about whether that procedure has actually been exercised more than once by more than one implementation, which is exactly what $C_{\text{Ver}} \rightarrow 1$ says has not happened. $\square$

This is the precise sense in which renderer and verifier concentration matter beyond restating a standard statistic: not as a threat to any theorem this essay has proven, all of which concern what the substrate makes *possible*, but as a standing gap between possible and actual corroboration that a reader relying on $A(b)$ or on several citations’ agreement should be able to check, per Section 9’s reader-rights framework, and that a namespace or platform operator concerned with $\text{Cap}(b)$ should have an incentive to close — diversity of verifier implementation is not a cosmetic property of a healthy ecosystem, on this account, but a precondition for citations’ agreement meaning what it appears to mean.

Finally, there is the risk of semantic drift through tooling. A renderer, summarizer, or recommendation system may gradually become the de facto authority through widespread use even when it was never intended to occupy that role. The canonical source remains available

in principle while becoming increasingly invisible in practice. The architecture can preserve provenance without guaranteeing that users will consult it.

These risks do not invalidate the proposal. They merely establish that source-first publishing should be understood as a redistribution of constraints rather than their elimination. The system exchanges the pathologies of artifact-centered publishing for a new family of problems associated with abundance, navigation, and governance. Recognizing those problems is part of designing responsibly rather than an argument against the design itself.

18 Semantic Objects Beyond Documents

Every worked example in this essay has been an essay, a paragraph, or a passage of argumentative prose, and a reader could reasonably conclude from that pattern alone that $\sigma = (I, T, M, E, S)$ is a document with extra bookkeeping attached — a well-typed, provenance-aware file format, and nothing more. Nothing in Definition 12.1 supports that conclusion, and it is worth showing concretely why not, rather than leaving the point implicit in the formalism and hoping a skeptical reader notices.

Definition 12.1 places no requirement on what T contains. A semantic object whose declared modalities are `PROSE-FULL` and `VIDEO` is a document in the sense this essay has mostly illustrated, but a semantic object whose declared modalities are `FORMAL-STATEMENT` and `INFORMAL-GLOSS` is a mathematical claim, and one whose modalities are `PREMISE-SET` and `DERIVATION` is a proof step, addressable and versionable by exactly the same machinery. A definition, in the sense of Appendix A’s `introduce_term` event, is already a semantic object in miniature: it has an identity, a narrow declared type, and a provenance graph of whatever reasoning motivated introducing it. A citation, per Section 14, is a semantic object whose content is a chain of rule applications rather than a passage of prose. A renderer is itself describable as a semantic object with its own identity and revision history, distinct from any particular σ it renders. A governance action — a change to the pointer-selection function P discussed in Section 13.3 — was already proposed there as “a first-class, versioned object subject to the same provenance discipline as everything else,” which is precisely a semantic object whose content is a decision rather than a claim. None of these required a new definition. They required only populating T differently.

This is the sense in which the architecture is not usefully described as version control for documents. A version-control system such as Git operates on files: opaque byte sequences whose diffs are syntactic, and whose semantic content — what a change actually asserts, proves, or governs — is invisible to the system doing the versioning. A semantic object’s identity I and provenance S persist independently of which modality happens to be populated, and its declared type T names what kind of thing it is before any content exists to diff. Where Git sees a changed line, this calculus can see a changed claim, a changed proof step, or a changed governance decision, each typed as such rather than reduced to a common denominator of text.

Read this way, Polyxan’s actual contribution to the stack described in Section 20 is not that it adds hyperlinks to documents. It is that the addressable node in the system was never a document in the first place; a document is one populated instance of T among many equally valid ones, and the essay’s repeated use of prose and video as worked examples reflects the motivating case of publishing, not a limit on what σ can represent. The architecture generalizes to any domain where claims, proofs, citations, and decisions need to be addressable, versioned, and traceable to their derivation — publishing is the case this essay develops in depth, not the boundary of what the formalism supports.

19 Addressability as a Primitive

Section 18 used the words *addressable*, *referenceable*, and *citable* throughout without pausing to justify why addressability matters in the first place. This section supplies that justification directly, since the claim is doing more work in this essay than its casual repetition might suggest.

An object that cannot be addressed cannot be cited, since a citation is, at minimum, a stable pointer that a second party can resolve independently of the first party’s continued cooperation. An object that cannot be cited cannot accumulate discourse around it, since discourse — agreement, rebuttal, extension, qualification — requires a fixed point that successive contributions can attach to without each contribution having to re-specify what it is responding to. Addressability is therefore not a convenience layered on top of a publishing system; it is a precondition for the kind of collective reasoning this essay has assumed throughout, from executable citations in Section 14 to the branch-disagreement queries of Section 9.

This dependency can be stated precisely once identity is separated from content, which Definition 12.1 already does by construction.

Lemma 19.1 (Addressability Lemma). *Let σ_t and σ_{t+1} be successive states of a semantic object related by a revision in the sense of Section 3, and let a citation to σ_t be a reference resolved through $I(\sigma_t)$ rather than through $M(\sigma_t)$ directly. If $I(\sigma_t) \neq I(\sigma_{t+1})$ — that is, if revision is permitted to change identity — then no citation issued against σ_t can be guaranteed to resolve to any determinate object once σ_{t+1} exists, since the identity a citation was anchored to no longer picks out a unique member of the object’s own lineage. Conversely, if $I(\sigma_t) = I(\sigma_{t+1})$ is preserved across every revision, as Definition 12.1 stipulates for I , then a citation issued against σ_t continues to resolve to a determinate node in S regardless of how many further revisions occur.*

Proof. A citation resolved through I rather than M is, by construction, a request to retrieve whichever node in the provenance graph S carries that identity at the time the citation was issued. If identity is not preserved across revision, the set of nodes carrying the identity referenced by the citation is not guaranteed to be a singleton at any later time, and resolution becomes ambiguous exactly to the degree that identity has drifted. If identity is preserved, per Definition 12.1, the node originally cited remains uniquely identified by I within S independently of how many further nodes have since been added, since S is append-only and I is immutable by hypothesis. $\square$

This is the sense in which I ’s immutability, stipulated almost in passing in Definition 12.1, is not bookkeeping but a load-bearing precondition. Everything this essay has said about citation surviving revision, in Section 5, and about executable citations resolving to a determinate derivation, in Section 14, depends on Lemma 19.1 holding, and Lemma 19.1 in turn depends on nothing more exotic than I never changing once assigned. One well-studied way of engineering that guarantee structurally, rather than by policy alone, is to derive identity from content itself through a cryptographic hash tree, so that identity is fixed by the data rather than by whichever party currently custodies it [9]; this essay does not commit to any particular addressing scheme, but the discipline such schemes formalize — identity that no single party can quietly reassign — is exactly what Lemma 19.1 requires of I , however it is implemented.

20 Situating the Proposal Within the Stack

20.1 Three Layers of the Stack, Briefly

A reader meeting SPHEREPOP, POLYXAN, and PLENUMHUB for the first time is owed a map before the detailed discussion below, since the three names arrive late in this essay and depend on

context this essay alone does not fully supply. At the coarsest level of description: Spherepop is the persistence layer, responsible for the fact that nothing is overwritten and every state is reachable from its history; Polyxan is the structure layer, responsible for content being addressable by identity and type rather than by file path, per Section 18's point that a semantic object need never be a document; and PlenumHub is the coordination layer, responsible for governance, attribution, and the social process of resolving disputes about which branch is shown by default, per Section 13.3. None of the three is a rebrand of an existing system under a new name — Polyxan is not Xanadu's hypertext with different terminology, Spherepop is not Git with a different diff format, and PlenumHub is not a forum with a different reputation score — but the precise sense in which each differs is the subject of the rest of this section, not something a one-paragraph map can establish on its own. What the map can establish is where this essay's proposal sits: source-first publishing, as developed in Sections 1 through 18, is the persistence and structure layers applied specifically to the case of publishing, with the coordination layer's open problems — named honestly rather than solved — inherited rather than newly introduced.

Source-first publishing is not a new project sitting alongside PLENUMHUB and POLYXAN; it is what those two projects already imply once applied to video, audio, and prose specifically. The three-layer stack this essay's proposal belongs to can be summarized as follows.

20.2 The Stack in Detail

Spherepop supplies event histories, collapse operations, provenance, and execution: the substrate-level fact that identity is an irreversible history of events rather than a persisting essence, and that nothing is overwritten, only extended. **Polyxan** supplies semantic structure and hypermedia: typed semantic objects rather than isolated documents, bidirectional linking, proof topology, and the claim — inherited explicitly from Project Xanadu [6] — that meaning itself, not merely text, should be addressable, versioned, and traversable in multiple directions. Media quines and sheaf-theoretic worldviews belong here: the requirement that locally reasonable interpretive acts glue together into a globally coherent semantic structure rather than silently contradicting one another at scale [7]. **PlenumHub** supplies coordination, governance, and economic and social infrastructure: identity as provenance and contribution rather than popularity, governance built on top of declared coherence conditions — stipulated rather than emergent, per Section 13.6's own correction of this essay's earlier phrasing — and, in the later, more explicitly thermodynamic framing, platform pathologies reframed not as bad incentives to be patched but as concentration of semantic capacity and curvature wells that attract attention, calling for redesign of the substrate itself rather than moderation layered on top of it.

All three have at various points been described as programs written in Spherepop calculus, and the recurring slogan across versions of this stack — *meaning as structured objects, identity as provenance, coherence as currency, time as persistence* — is a direct statement of what a canonical-source publishing layer needs to be true in order to work at all. A video generated on demand from a versioned source is a Polyxan object: typed, addressable, linked back to its lineage. Its branching revision history is a Spherepop event log: nothing overwritten, everything collapsed forward from a prior state. And the practice of citing a specific prior state, resolving disputes about which branch is canonical, or crediting a suggested continuation is a PlenumHub governance function: coordination and attribution handled as a property of the substrate rather than imposed by a platform's moderation team after the fact.

Seen this way, the complaint that opened this essay — that YouTube discards its source and treats an uneditable recording as canonical — was never really a complaint about video specifically. Conventional social media generally optimizes for feed-centric competition among posts for attention; the proposal here inherits, from the wider stack, an explicit rejection of that archi-

ture in favor of merge-aware, provenance-aware, event-sourced structures where correctness and coordination are properties of the substrate rather than outcomes fought for within it. The publishing problem turned out to be one more surface on which the same underlying commitment applies: that containers — histories, semantic structures, coordination mechanisms — are prior to and independent of their contents, and that a system built on that premise keeps a wide space of admissible futures open by construction, rather than collapsing it into whatever single frozen artifact happened to get rendered first.

20.3 Spherepop OS as an Instance of View–Cause Separation

The Spherepop OS specification referenced informally above has since been developed into a complete document: a four-primitive causal basis (Pop, Refuse, Bind, Collapse), three structural composition rules (sequence, tensor, Branch), and a full derived-function layer built from them, independently arrived at from an operating-systems starting point rather than a publishing one. What follows corrects and sharpens the informal parallels drawn earlier, now that both sides are fully formal rather than one being asserted against the other’s prose.

20.3.1 Collapse Is a Rendering Operation; Q Orchestrates Them

Proposition 20.1 (Collapse Corresponds to a Rendering Rule, Not to Closure). *Spherepop OS’s Collapse operator is an instance of this essay’s rule schema $r : a \rightsquigarrow b$ (Definition 12.3), specifically instantiating the rendering map ρ_k of Definition 12.23, not the closure operator Q .*

Proof. Both Collapse and ρ_k share the same signature shape: a map from a richer, history-bearing source (State terms carrying H_t ; provenance histories in $\mathcal{P}$) to a bounded, discardable output ($\text{Obs}; O_k$). Both are lossy in exactly that output: Collapse’s Obs value does not by itself determine which H_t produced it, and ρ_k is non-injective by Theorem 12.24. Neither operation destroys the underlying history in producing that output — H_t persists after Collapse exactly as S persists after any rendering, per Proposition 12.27 — so the loss is a property of the projection, not an erasure of its source. Q ’s signature and behavior match neither the shape nor the loss: $Q : \sigma \rightarrow \sigma$ is endomorphic, not type-changing, and it populates rather than discards. $\square$

This sharpens rather than merely restates an earlier informal claim. Q is not “the same kind of thing as Collapse, done differently” — Q is an endomorphic *orchestration* of potentially many individual rendering operations (each itself Collapse-shaped, lossy, State-to-bounded-output), applied once per declared modality until $\text{dom}(M) = T$. Collapse is one such operation, not the orchestrator. The true analogue of Q in Spherepop OS’s own vocabulary would be a procedure invoking Collapse once per fact key in a namespace’s authorized domain until every key is populated, terminating when Spherepop OS’s own reachability analogue holds and failing loudly rather than confabulating otherwise, per Definition 13.8. That orchestrating operator has no name yet in either document; naming it is left open.

20.3.2 Branch Is a Constructive Witness for Shared Ancestry, Not Its Only Source

Proposition 20.2 (Branch Constructively Witnesses Merge’s Precondition). *Definition 12.11’s hypothesis — that σ_a, σ_b “share a common ancestor in their provenance graphs” — is assumed, not constructed, anywhere in Section 12. Spherepop OS’s Branch judgment supplies one constructive witness for this hypothesis: a branch event, under its Prefix Coherence axiom, produces two histories provably agreeing up to a fixed point, which is a sufficient instance of “sharing a common ancestor,” stated as an enforced construction rather than presupposed as a property.*

Proof. Immediate from SpheroPOP OS’s Branch Prefix Coherence axiom’s guarantee that the two projections’ histories agree with the pre-branch history up to its length at the branch point: any pair so produced satisfies Definition 12.11’s ancestry hypothesis by direct inspection. $\square$

This is deliberately a weaker claim than a naive reading of the earlier connection would suggest. Branch is *a* way to construct the shared-ancestor relationship Definition 12.11 presupposes; it is not shown, and should not be read as claimed, to be the *only* way two objects could come to share an ancestor within this essay’s own calculus — ordinary sequential revision along the lineage axis of Section 3, for instance, could plausibly establish the same relationship without any branching event at all. What transfers is that this essay’s calculus currently has no constructive account of its own for this hypothesis, and Branch is an existence proof that one is possible, not evidence that Branch is required.

20.3.3 Guarded Merge and MERGE: Analogous, Not Identical

Proposition 20.3 (Structural Analogy, With Two Named Divergences). *Definition 12.11 ($\sigma_a \oplus \sigma_b$) and SpheroPOP OS’s MERGE reduction share a three-part shape — composing two objects into a joint context, applying an admissibility guard, and handling the guard’s failure — but diverge in the specific content of the first and third parts.*

Proof. Composition: this essay’s precondition is a hypothesis about σ_a, σ_b ’s existing relationship (shared ancestry), not an operation performed on them; SpheroPOP OS’s tensor is an operation, actively constructing a joint state from two possibly-unrelated ones. These are different kinds of things — a property versus a constructor — even where Proposition 20.2 shows Branch can supply instances satisfying both. Guard: both apply an admissibility filter (the entropy inequality here; a compatibility filter over composed histories in SpheroPOP OS), which is the point of genuine structural agreement. Failure handling: this essay’s $\oplus$ is a partial function, simply undefined when its guard fails, with no event recorded; SpheroPOP OS’s MERGE records a rejection event *on the success path*, marking the eliminated alternative (remaining separate) as part of history regardless of whether compatibility held. Undefined-and-silent is not the same mechanism as defined-and-recording-the-road-not-taken; the two essays make different choices about whether the rejected alternative becomes part of the historical record. $\square$

20.3.4 Identity Requires an Explicit Policy

Definition 12.13 and Proposition 12.14, stated earlier in this essay as a correction owed to Definition 12.11, were surfaced by exactly this comparison: SpheroPOP OS’s own proof that its structural tensor rule is silent on identity applies verbatim to $\oplus$, since the two operators share enough of their shape (Proposition 20.3) for the argument to transfer. SpheroPOP OS’s own resolution — a union-find update designating one input’s representative as canonical — is one particular instantiation of Φ_I , namely selection of one input under some externally fixed convention; stating Φ_I in general, as Definition 12.13 does, leaves room for policies that mint a fresh identity for the merged object instead, which neither document currently rules out or requires.

20.3.5 Namespace Coherence Sharpens This Essay’s Own Coherence Theorem

SpheroPOP OS independently needed, and proved, a negative result about multiple deterministic renderers of a shared source disagreeing — exactly the situation Theorem 12.17’s hypotheses do not, on close reading, rule out. That correction is recorded directly in Section 13.6 above, as hypothesis (iii): semantic equivalence of rules governing a shared modality across contexts, not necessarily one literal shared instance. It is repeated here only to note the direction of the finding — this is a case where SpheroPOP OS’s independently-motivated result exposed a gap

in this essay's own proof, not the reverse, and the essay is stronger for stating the gap rather than leaving Theorem 12.17 looking more complete than its proof actually established.

20.3.6 On the Reduction's Own Corrections

An earlier note flagged that Spheredpop OS's six kernel event types did not initially match its own four-primitive causal basis. That discrepancy has since been resolved, and resolved twice: POP and, originally, kernel COLLAPSE were both classified as directly primitive, until a comparison against this essay's own Definition 12.13 showed kernel COLLAPSE shares MERGE's identity-designation gap rather than being a clean instance of the causal Collapse primitive — kernel COLLAPSE mutates persistent union-find state, matching MERGE's type, not the type of a terminal, state-consuming rendering. That correction has been made directly in the Spheredpop OS document itself. The traffic between these two documents has therefore run in both directions: this essay's merge-identity machinery corrected an error in Spheredpop OS's own reduction table, and Spheredpop OS's namespace-coherence work corrected a gap in this essay's own coherence theorem. Two independently-derived formalisms converging this strongly, while each catching a real error in the other along the way, is a stronger form of confirmation than either document could have produced by internal review alone.

21 The End of the Document

The concept of a document is older than the digital systems that now distribute most human knowledge. A document assumes a bounded object with a beginning and an end, a stable sequence of contents, and a fixed identity independent of its production history. These assumptions were reasonable under the constraints of paper. They are no longer technologically necessary.

Much of the friction discussed throughout this essay can be understood as a consequence of preserving the document model long after the conditions that justified it disappeared. Version histories are hidden because documents are expected to be singular objects. Alternative continuations are difficult to represent because documents are expected to be linear. Corrections appear as amendments because documents are expected to be complete. The architecture inherited from paper continues to shape systems that no longer depend on paper.

A source-first architecture proposes a different primitive. The primary object is not a document but a history space. What persists through time is not a rendered artifact but a structured graph of events, transformations, and relationships — the object Definition 12.1 calls σ . Documents become temporary projections through that graph, generated for specific audiences, purposes, and contexts.

Seen from this perspective, essays, videos, podcasts, comics, lectures, and summaries occupy the same ontological status. None is canonical. None is the work itself. Each is a rendering of a deeper object whose identity is carried by provenance rather than presentation.

This shift parallels earlier transitions in computing. Source code displaced binaries as the primary object of software development. Databases displaced static reports as the primary object of information management. Version control displaced isolated snapshots as the primary representation of software history. In each case the enduring object moved from a rendered artifact to a structured process capable of producing many artifacts.

The proposal developed throughout this essay applies the same inversion to publishing. The goal is not to abolish documents. Documents remain useful because humans often prefer bounded narratives to open-ended structures. The goal is to relocate them from the foundation of the system to its surface. They become generated views rather than containers of truth.

That preference is not merely a habit left over from paper, and it would be a mistake for this essay to imply otherwise. Boundedness makes a work finishable, both to write and to read. A stable identity makes it citable without qualification. A fixed beginning and end make comprehension tractable in a way an open-ended graph, however well organized, does not automatically provide. These are cognitive facts about readers, not just technological facts about print, and a source-first architecture does not get to treat them as obsolete simply because the storage layer no longer requires them. What the architecture changes is where those properties live: a document remains bounded, stable, and readable in one sitting, but it becomes one closed projection through σ rather than σ itself, so that the cognitive convenience of a document and the corrigibility of a history are no longer purchased at each other's expense. The claim of this essay is that documents are useful projections of a history space, not that documents are obsolete; the difference matters, since only the first claim survives contact with why people wanted documents in the first place.

Put most compactly: the goal is not to abolish the document but to demote it, from ontology to interface. It stops being the thing that exists and becomes the thing a reader is shown — a real demotion, since only the source is retained, addressable, and correctable, but not an abolition, since every cognitive reason a reader ever had for wanting a document is still being served by the projection they receive.

If the architecture succeeds, the document does not disappear. It becomes what it perhaps always was: one path through a larger space of possibilities, valuable as a rendering but no longer mistaken for the source from which it came.

A A Minimal Event Grammar for Argumentative Prose

The worked example of Section 13.1 used a restricted event vocabulary sufficient for definitions, assertions, contrasts, and qualifications. This appendix gives that vocabulary in a compact BNF-style form. It formalizes the four operations already used in Section 13.1 and nothing beyond them; it remains, as Section 13.1 states, an existence proof for a bounded fragment of argumentative prose, not a general-purpose grammar.

A.1 Core Syntax

```
<event> ::= introduce_term
          | assert_relation
          | contrast
          | qualify

<introduce_term> ::=
  event {
    id: <event_id>,
    op: introduce_term,
    term: <term>
  }

<assert_relation> ::=
  event {
    id: <event_id>,
    op: assert_relation,
    subject: <reference>,
    relation: <relation>,
    object: <content>,
    depends_on: [<reference>*]
  }

<contrast> ::=
  event {
    id: <event_id>,
    op: contrast,
    terms: [<reference>+],
    depends_on: [<reference>*]
  }

<qualify> ::=
  event {
    id: <event_id>,
    op: qualify,
    target: <reference>,
    content: <content>,
    depends_on: [<reference>*]
  }
```

A.2 Auxiliary Nonterminals

```
<event_id> ::= identifier
```

`<reference> ::= <event_id>`

`<term> ::= string`

`<relation> ::= string`

`<content> ::= string`

A.3 Dependency Constraints

Six constraints govern any well-formed event history in this grammar. Every event identifier is unique, and every element of `depends_on` must reference an already-existing event, so that a dependency can never point forward into content that has not yet been introduced. An `assert_relation` event introduces a directed semantic edge from its subject to its object, giving the relation a fixed orientation rather than leaving it symmetric. A `contrast` event requires at least two referenced events, since a contrast drawn against a single term is not yet a contrast. A `qualify` event does not replace its target event but creates a new event whose interpretation is layered onto the target, preserving the target exactly as it was. And event histories are append-only throughout: existing events are never modified in place, which is what makes the diff in the worked example of Section 13.1 a single added node rather than a rewritten paragraph.

A.4 Dependency Graph Semantics

Let

$$G = (V, E)$$

be the event graph induced by a source object. Each event corresponds to a vertex $v \in V$, and each dependency $a \in \text{depends_on}(b)$ induces a directed edge $a \rightarrow b$. G is required to be acyclic, and a rendering order is any topological ordering of G .

A.5 Example

```
event { id: e1,
        op: introduce_term,
        term: "teacher" }

event { id: e2,
        op: introduce_term,
        term: "preacher" }

event { id: e3,
        op: assert_relation,
        subject: e1,
        relation: "increases",
        object: "capacity to evaluate multiple possibilities",
        depends_on: [e1] }

event { id: e4,
        op: assert_relation,
        subject: e2,
```

```

    relation: "secures",
    object: "commitment to a single viewpoint",
    depends_on: [e2] }

event { id: e5,
  op: contrast,
  terms: [e3,e4],
  depends_on: [e3,e4] }

event { id: e6,
  op: qualify,
  target: e5,
  content: "most educators do some preaching, and most
           preachers do some teaching",
  depends_on: [e5] }

```

A prose renderer traverses the graph in dependency order and maps each event type to a corresponding linguistic template. The resulting text is therefore a rendering of the event graph rather than the canonical source itself.

The grammar above specifies what an event is and how events depend on one another; it does not specify how a given event type is turned into English. That step is itself a rule in the sense of Definition 12.3 — a typed transformation from the event sequence to PROSE-FULL — and Definition 12.3 characterizes every rule in this calculus by its source and target type and its entropy budget, never by its internal computation. Two renderers may satisfy the same rule with different template libraries and produce different English for the same event graph; both are equally valid renderings under Definition 12.3, and choosing between them is a renderer design question this appendix leaves open by the same convention the calculus applies to every other rule, not by an oversight specific to prose.

B Notation and Definition Index

This index collects the symbols and named concepts introduced across this essay, in order of first appearance, for reference while reading the later sections that depend on them. It adds no content beyond what is already stated at each cited location.

Symbol / Term	Meaning	Location
$\sigma = (I, T, M, E, S)$	A semantic object: identity, declared modality tags, partial content map, entropy value, provenance graph.	Def. 12.1
I	Immutable identity of a semantic object.	Def. 12.1
T	Finite set of required (declared) modality tags.	Def. 12.1
M	Partial content map from modalities to payloads.	Def. 12.1
E	Scalar entropy value of σ .	Def. 12.1, 12.9
S	Provenance graph recording σ 's derivation history.	Def. 12.1
$\sigma \vdash \text{valid}$	Well-typedness: $\text{dom}(M) \subseteq T$.	Def. 12.2
$r : a \rightsquigarrow b$	A rule: a typed transformation from modality a to modality b .	Def. 12.3

Symbol / Term	Meaning	Location
ϵ_r	Entropy budget of rule r .	Def. 12.3
$\text{Reach}(\sigma)$	Modalities reachable from σ by some valid rule chain.	Def. 12.6
renderer-complete	σ such that $\text{Reach}(\sigma) = T$.	Def. 12.6
Q	Closure operator; populates every declared modality of a renderer-complete σ .	Def. 12.7
H, MI	Per-modality entropy and pairwise mutual information, used to define $E(\sigma)$.	Def. 12.9
$\sigma_a \oplus \sigma_b$	Guarded merge of two objects sharing a common ancestor.	Def. 12.11
$\epsilon_{\oplus}$	Merge budget bounding the entropy of a guarded merge.	Def. 12.11
Φ_I	External policy resolving $I(\sigma_a \oplus \sigma_b)$; not derivable from Def. 12.11 alone.	Def. 12.13
$G, U_i, \rho_{U_i U_j}$	Local view assignment over a covering of contexts, with restriction maps.	Def. 12.15
coherent	G satisfies pairwise agreement on overlaps and unique gluing.	Def. 12.16
$\mathcal{P}$	Provenance space: all provenance graphs reachable by finite rule chains.	Def. 12.22
ρ_k	Rendering map from provenance histories to outputs of modality k .	Def. 12.23
O_k	Space of practically bounded outputs for modality k .	Def. 12.23
erasure	Discarding S after producing $M(k)$, making it unavailable to later rules.	Def. 12.25
illusion	Inability to recover which provenance history produced an observed $M(k)$.	Def. 12.26
retentive / k -erasing	Policies that preserve S indefinitely, or discard it after populating k .	Def. 12.29
$S_{\text{recorded}}, S_{\text{actual}}$	The stored provenance graph versus the true derivation history.	Def. 13.2
Faithful(S)	$S_{\text{recorded}} = S_{\text{actual}}$.	Def. 13.2
unmarked rule	A transformation altering M that is absent from S_{recorded} .	Def. 13.3
confabulated closure	Content occupying a modality outside $\text{Reach}(\sigma)$, invented rather than derived or left absent.	Def. 13.8
verifiable rule	A rule whose input-output relation can be independently checked from $M(a)$ and $M(b)$ alone, without trusting the renderer.	Def. 13.6
$\text{True}(M)$	An external, reader-supplied correctness predicate the calculus does not define; used only to state that faithfulness and truth are independent.	§13.7
$P : \mathcal{B} \rightarrow \mathcal{B}$	Pointer-selection function choosing the default branch shown to a reader.	§13.3

Symbol / Term	Meaning	Location
$V(b), A(b), \text{Cap}(b)$	Exposure share, audit score, and capture for branch b .	Def. 13.1
$I(\sigma_t) = I(\sigma_{t+1})$	Addressability Lemma: identity preserved across revision is what lets citations resolve stably.	Lem. 19.1
$k_i \preceq k_j$	Refinement (depth) order: a deeper modality can always be compressed to a shallower one.	Def. 12.19
progressive disclosure	A pipeline lets a reader request any depth without first producing deeper ones.	Def. 12.20
$\pi(\sigma)$	Snapshot map: σ to its content map alone, discarding I, E, S .	Def. 6.1
V (no general verifier)	Any procedure deciding $\text{Faithful}(S)$ from S_{recorded}, M alone; shown not to exist in general.	Thm. 13.5
$\text{Admit}(\sigma)$	Admission-policy predicate; size-gating conflates acceptance with preservation/availability/discoverability.	Def. 12.31
$\Delta E(\sigma_{\text{new}} \mid C)$	Marginal compression contribution of a candidate object to a corpus.	Def. 12.32
$r@v_r, \text{hash}(r)$	An immutable, content-addressed rule version.	Def. 14.1
χ	= An executable citation: source, typed derivation	Def. 14.2
$(x_0, \delta_\chi, \text{VerifSpec}_\chi, \text{Ver}@_{\text{chain}})$	chain, verification spec, verifier payload.	
δ_χ	Typed derivation path $(r_1@v_1, \theta_1; \dots; r_n@v_n, \theta_n)$.	Def. 14.2
$P_{\text{claim}}, P_{\neg\text{claim}}$	Support and contradiction predicates over a citation's terminal output.	Def. 14.3
$\mathcal{V}$	Verification result type: Verified, Contradicted, Undetermined, Inexecutable.	Def. 14.6
$w(e), W(\delta)$	Structural salience of a provenance node / a chain, from its declared entropy budget.	Def. 17.2
$\eta = B/ S $	Inspectable fraction: reader budget over provenance-graph size.	Prop. 17.1
$s(\cdot)$	Verification score mapping $\mathcal{V}$ to $[0, 1]$.	Def. 17.7
C_R	Renderer concentration over empirical usage shares $p(r_i)$.	Prop. 17.13
C_{Ver}	Verifier-population concentration; $C_{\text{Ver}} \rightarrow 1$ discounts corroboration.	Def. 17.14

References

- [1] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [2] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [3] P. Villalobos, A. Ho, J. Sevilla, T. Besiroglu, L. Heim, and M. Hobbhahn. Will we run out of data? Limits of LLM scaling based on human-generated data. *arXiv preprint*

arXiv:2211.04325, 2022 (revised 2024).

- [4] D. Jiang, M. Wang, H. Yang, and J. Wang. From SRA to Self-Flow: data augmentation or self-supervision? *arXiv preprint arXiv:2607.02508*, 2026.
- [5] W. Zhang, L. Hotsko, W. Kim, P. Nie, S. Shieber, and Y. Deng. Program-as-Weights: a programming paradigm for fuzzy functions. *arXiv preprint arXiv:2607.02512*, 2026.
- [6] T. H. Nelson. Xanalogical structure, needed now more than ever: parallel documents, deep links to content, deep versioning, and deep re-use. *ACM Computing Surveys*, 31(4es), 1999.
- [7] S. Mac Lane and I. Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Springer-Verlag, 1992.
- [8] P. Buneman, S. Khanna, and W.-C. Tan. Why and where: a characterization of data provenance. In *Database Theory — ICDT 2001*, Lecture Notes in Computer Science vol. 1973, pages 316–330. Springer, 2001.
- [9] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, Lecture Notes in Computer Science vol. 293, pages 369–378. Springer, 1987.
- [10] P. Groth, A. Gibson, and J. Velterop. The anatomy of a nanopublication. *Information Services and Use*, 30(1–2):51–56, 2010.
- [11] E. Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.